PROF. DR.-ING. MARK SCHUTERA

_SUPERVISED LEARNING

UNFINISHED LECTURE NOTES

Copyright © 2026 Prof. Dr.-Ing. Mark Schutera

PUBLISHED BY UNFINISHED LECTURE NOTES

Combobulated with the help of multiple large language model driven tools. Licensed under the Creative Commons Attribution-NonCommercial 4.0 International License ("CC BY-NC-SA 4.0"). You may not use this file for commercial purposes. If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original You must obtain explicit permission from the author for uses beyond those permitted by this license. To view a copy of this license, visit https://creativecommons.org/licenses/by-nc-sa/4.0/. Unless required by applicable law or agreed to in writing, distributed material is provided on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the license for details.

Last updated, April 2026 — v0.0.10

"IN MY DEFENSE, I WAS LEFT UNSUPERVISED".

DER ZAUBERLEHRLING

Contents

I Geometry of Pattern 11

II Representing Pattern 13

Variational Inference 15

Uncertainty Estimation 39

III Epistemic Pattern 53

Self-Supervised Learning 55

Index 69

Introduction

Supervised learning is limited to tasks where we have labeled data and clear objectives. Beyond this limited context of learning, we fall back to the pattern in the data itself.

We explore how pattern emerges, from other sources of information beyond explicit labels. We will see how pattern can be discovered in data, represented, leveraged and utilized. In that sense this course is about any form of learning that is not supervised learning.

Each lecture follows a consistent pattern. To start of the contents of the lecture will be motivated by a narrative, a theoretical primer, and some sort of hands-on experience to build intuition. Then we will dive into the, theory and algorithms, then reflection. Theory sessions are 45 minutes, followed by 45 minutes of coding practice to enforce reflection and deepen understanding of the concepts.

A Philosophy of Education

The beautiful risk of education is that it is an encounter. I am trying to model a way of thinking, and reasoning, then stepping back. The rest is yours.

- **Start with why.** A method understood only procedurally is a method you will misapply when it matters. Before learning how something works, understand why it was built, what it replaced, and where it fails. The mechanics then follow from necessity, not memorization.
- **Build taste alongside competence.** You need a sense of what beautiful is, you need to train your aesthetics before you can reliably produce it. This develops by studying what others have built, making it your own, then making your own, and honestly evaluating the gap between them. Taste is the compass. Skill is the engine. You need both, in that order.
- **Failure is the designed medium.** Each attempt teaches what a clean run cannot. The insight compounds across tries, not from a single correct execution. Those who never start failing will fail.
- **Work in the open.** Showing half-finished thinking, naming mistakes as they happen, is a professional skill. We practice it here. It is how knowledge actually moves between people. And it is how knowledge moves towards you, at unprecedented speed.
- **The fallacy of mediocracy.** Iteration and failure are the method, not the excuse for settling. Do not confuse velocity with quality. Whatever you learn, whatever you build: make it good.
- **Outcomes emerge. They are not delivered.** Judgment, taste, the capacity to sit with uncertainty: these are not course outputs. They are what you build from the encounter. I can create the conditions. The rest depends on what you do with them.

Part I

Geometry of Pattern

Part II

Representing Pattern

Variational Inference

EMBEDDINGS AND LATENT FEATURE SPACES.

The Why

THE CENTRAL PROBLEM OF UNSUPERVISED LEARNING is representation learning: finding a mapping from raw observations to a latent feature space in which the underlying factors of variation become explicit. Bengio, Courville, and Vincent[1] argue that the quality of a representation determines the success of any downstream task, and that a good representation disentangles the factors that explain the data.

PLATO'S CAVE ALLEGORY describes prisoners who see only shadows on a wall, never the objects that cast them. The shadows are high dimensional and entangled; the true forms behind them are few and independent. Dimensionality reduction, as we have seen, tries to recover those forms from the shadows. This chapter asks two follow-up questions: can we organise the recovered forms into a structured, reusable feature space, and once we have that space, can we cast new, plausible shadows we have never observed?

Plato. *The Republic*. Penguin Classics, 2007. Book VII, 514a–520a. Originally written c. 380 BCE

PCA gave us a first answer: project onto the directions of maximum variance. But PCA is linear; it cannot recover structure that lies on a curved manifold. What we lack is a parametric encoder that can be applied to new, unseen data or reused as a feature extractor on nonlinear feature spaces.

AUTOENCODERS close that gap. Hinton and Salakhutdinov[2] showed that a neural network can learn a nonlinear, low-dimensional embedding z from which a second network reconstructs $\hat{x}$, outperforming PCA on complex data. The encoder $\theta_E \colon \mathbb{R}^d \to \mathbb{R}^k$ and the decoder $\theta_D \colon \mathbb{R}^k \to \mathbb{R}^d$ are trained jointly to minimise reconstruction error $\|x - \theta_D(\theta_E(x))\|^2$. The bottleneck forces the network to discover a compact representation.

STRUCTURE REQUIRES A PROBABILISTIC FORMULATION.

If we replace the deterministic bottleneck with a distribution, and regularise that distribution toward a known prior, the latent space becomes smooth and complete: every region maps to a plausible output, and nearby embeddings correspond to similar inputs. Kingma and Welling[3] formalised this idea as the Variational Autoencoder, deriving a principled training objective from variational inference that

[1] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013. DOI: 10.1109/TPAMI.2013.50. https://scholar.google.com/scholar?q=Bengio+Courville+Vincent+2013+representation+learning+review+new+perspectives

[2] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. DOI: 10.1126/science.1127647. https://scholar.google.com/scholar?q=Hinton+Salakhutdinov+2006+reducing+dimensionality+neural+networks

A PRIOR is the distribution we assume over the latent space before seeing any data.

[3] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014. https://scholar.google.com/scholar?q=Kingma+Welling+2014+Auto-Encoding+Variational+Bayes

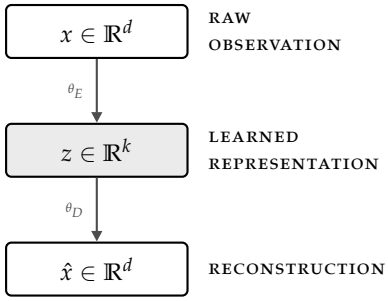


Figure 1: Autoencoder architecture. The encoder θ_E maps the input to a low-dimensional representation; the decoder θ_D reconstructs the input from it. The bottleneck forces the network to discover a compact latent feature space.

balances faithful reconstruction against latent space regularity.

ALL OF THIS MUST HAPPEN WITHOUT LABELS. Supervised learning can organise a feature space by asking "which class does this input belong to?" and adjusting representations until the answer is easy to read off. An unsupervised model has no such signal. Instead, it must derive structure from the data alone, using two complementary pressures: the requirement to reconstruct the input faithfully, and the requirement to keep the latent space close to a known prior. Reconstruction forces the embedding to retain information; the prior forces the embedding to be organised. Neither pressure requires a human to annotate a single example.

IN ORDER TO MOVE FROM RECONSTRUCTION-ONLY EMBEDDINGS TO WELL-ORGANISED LATENT FEATURE SPACES we have good reason to find ways to,

- formulate a learning objective that rewards both faithful reconstruction and a smooth, regular latent space.
- train a model that outputs distributions rather than points, so that the latent space has no undefined gaps.
- control how much structure the latent space carries, and understand how that choice affects the quality of the learned representation.

Hands-On Experience

FORM GROUPS OF TWO: an encoder and a decoder. Prepare five digit cards showing **0**, **1**, **3**, **7**, and **8**. Shuffle them face down on the table.

STEP 1: DESIGN YOUR FEATURES. The encoder picks two properties of digit shapes and writes them down. These are your two latent features $z^{(1)}$ and $z^{(2)}$. For example: $z^{(1)}$ = "number of closed loops" (0 has one, 8 has two, 1 has none) and $z^{(2)}$ = "has a straight vertical stroke" (1 yes, 7 yes, 0 no); make sure to come up with your own The decoder does not see these definitions.

SCORING. After the decoder guesses, flip the card. Correct digit = 0, wrong digit = 1. Sum over all rounds: that is your reconstruction loss.

STEP 2: PLAY. Each round, the encoder flips one card, looks at the digit, and writes two numbers $(z^{(1)}, z^{(2)})$ on a slip of paper. The decoder receives only the slip and guesses which digit it is. Then reveal the card. The decoder now knows: "the embedding $(1, 0)$ meant the digit 3." Round by round, the decoder builds a mental map of what the embeddings mean. When all five cards are done, shuffle and keep going.

STEP 3: OBSERVE. In early rounds, the decoder guesses blindly. With each reveal, the decoder learns what the embeddings mean. At some point the reconstruction loss drops to zero: the decoder has learned the encoder's representation from examples alone.

STEP 4: RECORD AND COMPARE. Plot your five embeddings in the coordinate system below.

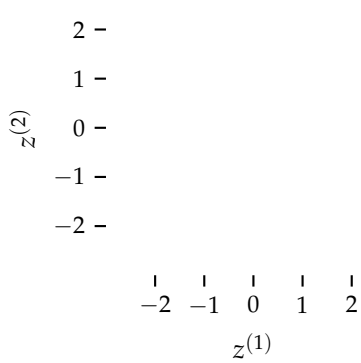


Figure 2: Your latent space. Plot the five digit embeddings at the coordinates $(z^{(1)}, z^{(2)})$ your encoder chose.

STEP 5: SWAP DECODERS. Find another group. Send your decoder to them, take theirs. Play one more pass with the swapped decoder.

- How many rounds did the decoder need before reconstructions became reliable?
- Which digits did the decoder confuse most often? What does that tell you about your feature choice?

The swapped decoder has learned a different codebook: in their group, $(1, 0)$ might have meant "7," in yours it means "3." The reconstruction loss jumps back up. The problem is not that either representation is wrong; both worked within their own group. The problem is that nothing forced the two groups to organise their embeddings the same way.

STEP 6: SHRINK THE BOTTLENECK. Drop to a single latent feature $z^{(1)}$. The encoder must pick the one property that matters most; the decoder has only one number to work with. Play a full pass and record the new reconstruction loss.

STEP 7: DISCRETISE. Go back to two features, but restrict each to binary values (0 or 1). The encoder can no longer spread digits across a continuous range and must find features that cleanly partition the digit set with just two bits.

STEP 8: ADD DIGITS. Keep two binary features, but add the cards **2**, **4**, **5**, and **9**. A representation that separated five digits may collapse when it must also distinguish eight. But the extra classes also expose ad hoc feature choices: properties that happened to work for a small set get overwhelmed, and the encoder is pressured toward genuinely structural features (loops, strokes, intersections) that scale.

STEP 9: SWAP DECODERS AGAIN. Find a different group than in Step 5. Exchange decoders and play one pass with the expanded digit set. Compare the reconstruction loss to the Step 5 swap: did the stress tests in Steps 6 to 8 push both groups toward more compatible representations.

After each decoder swap, collect the chosen latent features from all groups on the board. Are the feature sets converging across the room? Which features survived the stress tests and which were abandoned?

EACH STEP HURTS RECONSTRUCTION but forces the encoder to find features that generalise across inputs rather than overfit to a narrow set. This tension between reconstruction fidelity and generalisation through compression or broader coverage is exactly the trade-off that the VAE formalises. And a trade-off you know from supervised learning as well, overfitting versus generalization.

THIS IS THE REPRESENTATION LEARNING PROBLEM. Every choice of features defines a different geometry: different neighbours, different distances. Within one encoder-decoder pair, reconstruction works. Across pairs, the embeddings are incompatible.

TWO OBJECTIVES, ONE GAME. Steps 2 to 4 trained the reconstruction part: the decoder learned to recover digits from embeddings. Steps 5 and 9 exposed the missing regularisation: without a shared convention for what the embeddings mean, a new decoder cannot read them. Steps 6 to 8 showed that compression and broader coverage force better features. A VAE adds exactly this second pressure: it forces every encoder to arrange its embeddings according to a shared prior, so that any decoder can use them.

THE FUNDAMENTAL QUESTION is: can we add a shared convention to the latent space, so that the embeddings are not only good for

reconstruction but also follow a common structure that any decoder could interpret?

THE LEARNING OBJECTIVES of this lecture:

- The Autoencoder Training Cycle. Architecture, forward pass, reconstruction loss, backpropagation, and what the bottleneck forces the network to learn.
- From Autoencoders to VAEs. Why reconstruction alone leaves the latent space unstructured, and what a probabilistic formulation buys us.
- The Evidence Lower Bound. The training objective that balances faithful reconstruction against latent space regularity.
- The Reparameterisation Trick. How to backpropagate through a stochastic sampling step.
- VAE Training and Bottleneck Design. Practical choices that shape the latent space: loss functions, latent dimensionality, and numerical stability.
- Exploring the Latent Space. Interpolation and generation as tests of representation quality.
- VAE Variants. How they each change one structural assumption.
- Feature Extraction, Transfer, and Distillation. Using the trained encoder as a backbone for downstream tasks.

The Autoencoder

Follows the notion of a bottleneck architecture: an encoder that compresses the input into a low-dimensional embedding, and a decoder that reconstructs the input from that embedding. Due to the bottleneck, the encoder must learn to extract the most salient features of the input, and the decoder must learn to use those features to reconstruct the input as losslessly as possible.

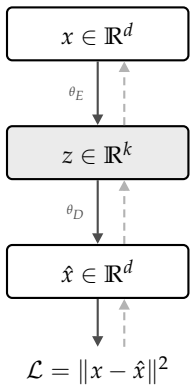


Figure 3: Autoencoder training cycle. Solid arrows: forward pass ($x \to z \to \hat{x} \to \mathcal{L}$). Dashed arrows: gradient flow back through decoder and encoder.

THE ENCODER $\theta_E \colon \mathbb{R}^d \to \mathbb{R}^k$ compresses a d-dimensional input x into a k-dimensional embedding z, with $k \ll d$.

NOTATION. θ_E denotes the encoder; θ_D denotes the decoder. Both are neural networks trained jointly.

THE DECODER $\theta_D \colon \mathbb{R}^k \to \mathbb{R}^d$ takes z and attempts to reconstruct the original input as $\hat{x}$.

THE FORWARD PASS maps a single training example through both networks. Given an input x, the encoder produces the embedding:

$$z = \theta_E(x)$$

The decoder then reconstructs:

$$\begin{aligned} \hat{x} &= \theta_D(z) \\ &= \theta_D(\theta_E(x)) \end{aligned}$$

BACKPROPAGATION is the backbone of supervised learning, and it applies here unchanged. The loss $\mathcal{L}$ depends on θ_D (through $\hat{x}$) and on θ_E (through z, which feeds into $\hat{x}$). The chain rule gives:

$$\frac{\partial \mathcal{L}}{\partial \theta_D} = \frac{\partial \mathcal{L}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial \theta_D}$$

for the decoder, and

$$\frac{\partial \mathcal{L}}{\partial \theta_E} = \frac{\partial \mathcal{L}}{\partial \hat{x}} \cdot \frac{\partial \hat{x}}{\partial z} \cdot \frac{\partial z}{\partial \theta_E}$$

for the encoder. The gradient signal flows from the loss, through the decoder, through z, and into the encoder. Both networks are updated from the same single loss.

THE RECONSTRUCTION LOSS measures how accurate the decoder reconstructs the input. The standard choice is the mean squared error over all dimensions:

$$\begin{aligned} \mathcal{L}(\theta_E, \theta_D; x) &= \|x - \hat{x}\|^2 \\ &= \sum_{j=1}^{d} (x_j - \hat{x}_j)^2 \end{aligned}$$

THE PARAMETER UPDATE applies standard gradient descent (or a variant such as Adam):

$$\theta_E \leftarrow \theta_E - \eta \frac{\partial \mathcal{L}}{\partial \theta_E}$$

$$\theta_D \leftarrow \theta_D - \eta \frac{\partial \mathcal{L}}{\partial \theta_D}$$

where η is the learning rate. One forward pass, one backward pass, one update: that is a single training step. Repeating this over minibatches and epochs, the encoder and decoder co-adapt until the reconstruction error converges.

THE BOTTLENECK. Because $k \ll d$, the embedding z cannot simply copy x. The encoder must decide which aspects of the input to preserve and which to discard. This forced compression is what generalizes the encoder into a feature extractor.[4]

THE TRAINED ENCODER IS A FEATURE EXTRACTOR, OR DIMENSIONALITY REDUCER. Once training converges, the decoder can be discarded. The encoder alone maps any new input x to an embedding z that captures the factors of variation learned during training. This embedding can serve as input to a classifier, a clustering algorithm, or a retrieval system. The autoencoder has learned a representation; the question is whether that representation is generalized.

$$x \in \mathbb{R}^d \xrightarrow{\theta_E} z \in \mathbb{R}^k$$

CONNECTION TO PCA. If the encoder and decoder are both linear (no activation functions) and the loss is MSE, the autoencoder recovers the same subspace as PCA[5]. The k embedding dimensions span the top-k principal component directions. Adding nonlinear activations allows the network to capture structure that PCA cannot: curved manifolds, discrete clusters, hierarchical features.

[4] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006. DOI: 10.1126/science.1127647. https://scholar.google.com/scholar?q=Hinton+Salakhutdinov+2006+reducing+dimensionality+neural+networks

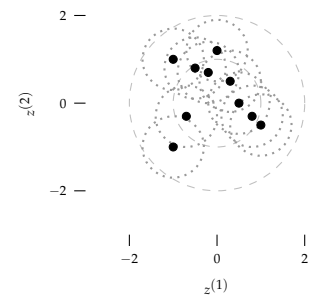
WHY DEEP (LEARNING)? Each layer applies a linear transformation followed by a nonlinear activation. A single hidden layer can in principle approximate any function, but may need impractically many neurons and is hard to learn. Deeper networks compose simple nonlinearities into increasingly abstract features: the first layer detects edges, the second combines edges into textures, the third recognises parts. Each layer reuses the representations below it, so the network expresses complex structure with far fewer parameters than a shallow one. Hinton and Salakhutdinov showed that a deep autoencoder with the same total capacity as a shallow one produces qualitatively better embeddings, because hierarchical composition is a more efficient way to represent the kind of structure found in real data.

[5] P. Baldi and K. Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2(1):53–58, 1989. DOI: 10.1016/0893-6080(89)90014-2. https://scholar.google.com/scholar?q=Baldi+Hornik+1989+neural+networks+principal+component+analysis

From Autoencoders to Variational Autoencoders

Standard autoencoders learn $x \to z \to \hat{x}$, minimising $\|x - \hat{x}\|^2$.

The latent space has no geometric guarantees. The objective only rewards reconstruction; nothing constrains how embeddings are arranged. Nearby points may decode to unrelated outputs, gaps between embeddings are undefined, and the layout can change arbitrarily across training runs. The representation captures enough to reconstruct, but it does not organise the factors of variation in a way that supports interpolation, sampling, or transfer.



VAE latent space. Each small circle is one input's encoder distribution $q_{\theta_E}(z|x_i)$; the dot marks its mean μ. The large dashed circles show the prior $p(z) = \mathcal{N}(0, I)$ at 1σ and 2σ. The KL term pushes each small circle toward unit variance and its mean toward the origin, so that the mixture of all encoder distributions approximates the prior.

To fix this, we add a constraint. In a plain autoencoder, the encoder's output layer produces a single vector $z \in \mathbb{R}^k$: one fixed point per input. In a Variational Autoencoder, the output layer produces two vectors instead: a mean $\mu \in \mathbb{R}^k$ and a variance $\sigma^2 \in \mathbb{R}^k$. Together they define a Gaussian distribution over the latent space:

$$q_{\theta_E}(z|x) = \mathcal{N}(\mu, \sigma^2)$$

During training, we do not pass μ directly to the decoder. Instead we sample from that distribution:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

So the decoder sees a slightly different z every time, even for the same input.

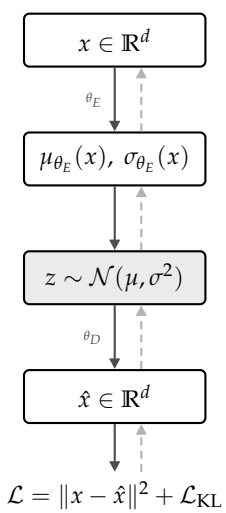
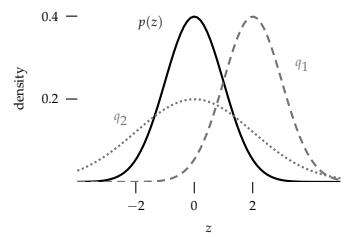


Figure 4: VAE architecture. Compare with Figure 3: the deterministic bottleneck is replaced by distribution parameters μ, σ and a stochastic sample. The loss combines reconstruction fidelity with KL regularisation toward the prior.

We enforce that by adding a loss a KL divergence $\mathcal{L}_{\mathrm{KL}}(q_{\theta_E}(z|x) \| p(z))$, computed for each input separately. It measures how far that input's

encoder distribution $\mathcal{N}(\mu, \sigma^2)$ is from the prior $p(z) = \mathcal{N}(0, I)$. The KL between two Gaussians penalises two things: a mean μ that is far from the origin, and a variance σ^2 that deviates from one. So for each input, the KL term pulls the mean toward zero and the variance toward unity.



KL PENALTY. $p(z) = \mathcal{N}(0, 1)$ is the prior (solid). $q_1 = \mathcal{N}(2, 1)$ (dashed) is shifted; $q_2 = \mathcal{N}(0, 4)$ (dotted) is widened. Spreading mass (q_2, $\mathcal{L}_{\text{KL}}$=0.81) costs more than translating it (q_1, $\mathcal{L}_{\text{KL}}$=0.50, evaluated in Exercise 2).

IF THE KL TERM ACTED ALONE, every input would collapse onto the same distribution $\mathcal{N}(0, I)$, and the decoder could not tell any two inputs apart. The reconstruction term fights back: it needs different inputs to have distinguishable embeddings, so it pushes their means apart. Training finds a compromise. Each input gets a distribution that is close to the prior but not identical to it: the means spread out just enough for the decoder to reconstruct, and the variances stay close to one. When all these slightly shifted, roughly unit-variance Gaussians are added together, their aggregate approximates the prior. That is how the prior fills in the gaps between embeddings and gives the latent space its geometric structure. Together they turn the encoder into a feature extractor whose embeddings are both informative and geometrically well behaved: nearby embeddings correspond to similar inputs, and every region of the space is meaningful.

THE RESULT is a latent space where geometric proximity reflects semantic similarity.

VAE Variants

β-VAE[6] scales up the KL term to pressure the encoder to use the latent space efficiently. The loss reweights the with a single hyperparameter:

$$\mathcal{L}_\beta = \text{Reconstruction} + \beta \cdot \mathcal{L}_{\text{KL}}$$

VQ-VAE[7] replaces continuous latents with discrete codes from a learned codebook. Each encoder output z_e is snapped to the nearest codebook entry, giving reconstructional structure:

$$z_q = \arg \min_{e_k \in \text{codebook}} \|z_e - e_k\|$$

CONDITIONAL VAE[8] feeds a conditioning variable c, such as a class label, into both encoder and decoder. The model now learns conditional distributions, which enables directed generation:

$$q_{\theta_E}(z|x, c), \quad p_{\theta_D}(x|z, c)$$

[6] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. β-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations (ICLR)*, 2017. https://scholar.google.com/scholar?q=Higgins+2017+beta-VAE+learning+basic+visual+concepts

[7] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://scholar.google.com/scholar?q=van+den+Oord+2017+neural+discrete+representation+learning+VQ-VAE

[8] K. Sohn, H. Lee, and X. Yan. Learning structured output representation using deep conditional generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015. https://scholar.google.com/scholar?q=Sohn+Lee+Yan+2015+learning+structured+output+conditional+generative

The Reparameterisation Trick

How do we now the distribution of the current embedding space? Our model now reconstructs data in two steps: draw z from the prior, then decode. To measure how well the model explains an observation x, we would need to sum over all possible embeddings z and check which ones decode close to x:

$$p_{\theta_D}(x) = \int p_{\theta_D}(x|z)\, p(z)\, dz$$

In words. The probability that the model generates x equals, summed over every possible latent z, the probability that the decoder produces x given that z, weighted by the prior probability of drawing that z in the first place. Each z is one possible explanation; the integral combines all of them.

Spoiler. We can't. This integral has no closed form and the latent space is too high-dimensional to sum over by brute force. Instead of summing over all of latent space, the encoder $q_{\theta_E}(z|x)$ output is used as an estimate, for each input x, where useful embeddings z for x live. We sample only from that region[9], turning an impossible integral into a practical expectation over a handful of samples.

[9] D. P. Kingma and M. Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2014. https://scholar.google.com/scholar?q=Kingma+Welling+2014+Auto-Encoding+Variational+Bayes

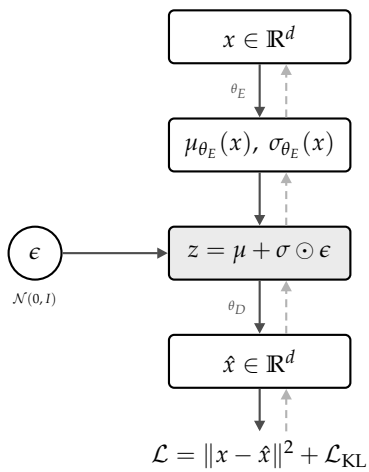


Figure 5: Reparameterised VAE or **Kingma's view**. Compare with Figure 4: the stochastic sample is replaced by $z = \mu + \sigma \odot \epsilon$ where ϵ carries the randomness and ,is drawn from a fixed distribution. Gradients (dashed) flow through μ and σ without passing through the random sample.

Designing the Bottleneck

THE LATENT DIMENSIONALITY k is the most consequential hyperparameter in a VAE. It controls the capacity of the information channel between encoder and decoder: how many bits of information about x can pass through z.

$$z \in \mathbb{R}^k$$

The bottleneck embedding. The dimensionality k controls how much information passes from encoder to decoder.

TOO FEW DIMENSIONS and the encoder cannot represent the factors of variation in the data. Reconstruction quality degrades because the embedding lacks the capacity to distinguish meaningfully different inputs. Two inputs that differ in important ways may map to nearly identical latent vectors, and the decoder produces a blurred average of what those embeddings could mean.

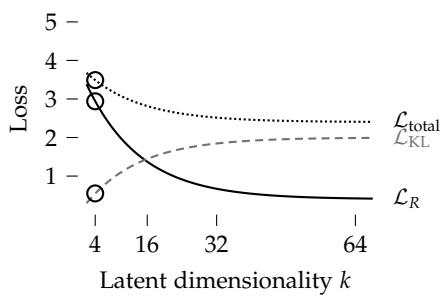


Figure 6: Bottleneck too small. At k=4, reconstruction loss is large ($\mathcal{L}_R \approx 2.9$) because the few active dimensions cannot encode all factors of variation. Each active dimension carries a lot of signal, but only a handful of them are paying KL ($\mathcal{L}_{\text{KL}} \approx 0.55$). The total (○ on the dotted curve) sits high because reconstruction error dominates.

TOO MANY DIMENSIONS and the model faces the opposite problem. With excess capacity, the encoder can encode each training point precisely without needing to discover shared structure. The KL term penalises unused dimensions toward the prior, but is blended out by the reconstruction success.

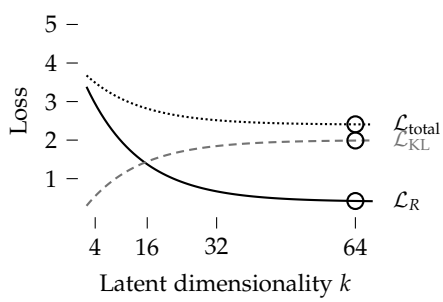


Figure 7: Bottleneck too large. At k=64, reconstruction loss has saturated near its floor ($\mathcal{L}_R \approx 0.42$) and the KL cost has saturated near its ceiling ($\mathcal{L}_{\text{KL}} \approx 1.99$): adding more dimensions changes neither term, because extra dimensions collapse to the prior and pay no KL. The total (○ on the dotted curve) sits at the plateau. The cost of k too large is therefore wasted parameters and slower training, not a higher loss.

THE INFORMATION BOTTLENECK PRINCIPLE[10] provides a theoretical lens. The optimal embedding retains all information in x that is relevant to reconstruction. The latent dimensionality k sets the ceiling

[10] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *arXiv preprint physics/0004057*, 2000. https://arxiv.org/abs/physics/0004057

on how much can be stored; the KL weight determines how aggressively the model compresses toward that ceiling to discarding weak representations.

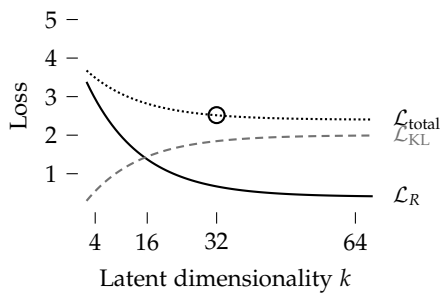


Figure 8: Bottleneck sizing trade-off. Reconstruction loss (solid) falls as k grows; the KL cost (dashed) rises until the data's information has been encoded, then plateaus as additional dimensions collapse to the prior. The total (dotted) is L-shaped: it drops sharply, then flattens. A good k sits near the elbow (○ at k=32), where the marginal gain per added dimension has fallen below what the extra parameters and training time cost.

Active units. Inactive dimensions have collapsed to the prior, as there is no potential reconstruction gain contributed by their information. Monitoring the number of active units during training reveals how much of the latent capacity the model actually uses[11].

Active-unit criterion. A latent dimension j is called active if

$$\mathcal{L}_{\text{KL}}(q(z_j|x) \,\|\, p(z_j)) > \delta$$

for a meaningful fraction of the training data.

[11] Y. Burda, R. Grosse, and R. Salakhutdinov. Importance weighted autoencoders. In *International Conference on Learning Representations (ICLR)*, 2016. https://arxiv.org/abs/1509.00519

This reading only holds in a particular regime. The active-unit count is meaningful when k is rather large that some dimensions could afford to drop, and when the reconstruction loss is competitive with the KL pull. The diagnostic is really a measure of how many dimensions the reconstruction loss can keep alive against the KL pull. So cross-dimension activity comparisons are needed.

A practical protocol for choosing k:

- Start with a generously large k. Adding capacity rarely hurts validation reconstruction; it just stops helping past the elbow.
- Train once and read off the effective capacity. The number of active dimensions is a direct estimate of the data's information ceiling.
- Use validation reconstruction, not training loss, as the model-selection signal to be sensitive to overfitting.

Rule of thumb. For input dimensionality d, a reasonable starting point is k on the order of $\sqrt{d}$ for tabular and pixel data. Text is a special case: vocabulary size makes d huge (30k+ tokens), but semantic information for sentence-level text VAEs, start at $k \approx 32-128$.

Two exceptions where the protocol does not apply. For visualisation, set k=2 or k=3 outright and accept the fidelity loss; the goal is a readable scatter, not optimal reconstruction. For very large models, where a single training run is expensive, pick a k from a comparable published architecture and skip the sweep entirely.

Beyond dimensionality, the structure of the prior itself shapes what the latent space can express. The standard choice $p(z) = \mathcal{N}(0, I)$ assumes the latent factors are independent and identically scaled. When the true factors of variation have different scales, correlations, or discrete structure, a mismatched prior forces the encoder to waste capacity compensating for the mismatch.

KL annealing ramps the KL weight from zero over the first N epochs, letting the encoder learn useful representations before regularisation kicks in and prevents posterior collapse.

S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, pages 10–21, 2016. https://arxiv.org/abs/1511.06349

Exploring the Latent Space

LATENT INTERPOLATION reveals the structure that the VAE has learned. Given two data points x_1 and x_2, encode, interpolate in the latent space, then decode each step:

$$
\begin{aligned}
z_1 &= \mu_{\theta_E}(x_1) \\
z_2 &= \mu_{\theta_E}(x_2) \\
z_t &= (1-t)\, z_1 + t\, z_2 \\
\hat{x}_t &= \theta_D(z_t)
\end{aligned}
$$

Because the latent space is regularised, the decoded path transitions smoothly between the two inputs[12]. Smooth interpolation is the simplest test of whether the encoder has learned a genuinely continuous feature space rather than a scattered collection of embeddings.

[12] T. White. Sampling generative networks. *arXiv preprint arXiv:1609.04468*, 2016. https://scholar.google.com/scholar?q=White+2016+Sampling+Generative+Networks

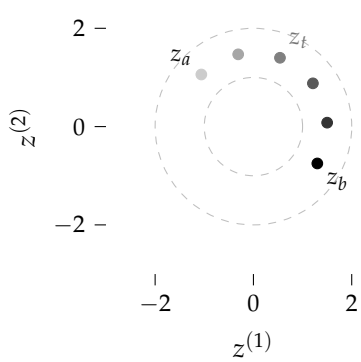


Figure 9: Latent interpolation. Two inputs encode to z_a and z_b. Linear interpolation (lerp, dashed) cuts through the low-density middle of the prior; in high dimensions this region is empty under $\mathcal{N}(0, I)$ and decoded midpoints look unnatural. Spherical linear interpolation (slerp, solid arc) follows the high-density shell at roughly $\|z\| \approx \sqrt{d}$, keeping every z_t on-distribution. Each waypoint is decoded to produce a smooth transition.

GENERATION is equally straightforward: sample from the prior and decode.

$$
\begin{aligned}
z &\sim \mathcal{N}(0, I) \\
\hat{x} &= \theta_D(z)
\end{aligned}
$$

The regularisation ensures that every region of the prior generates plausible data. The same property that makes the encoder a reliable feature extractor, a latent space with no dead zones, is what makes generation possible.

WELCOME TO GENERATIVE AI. A trained decoder paired with a tractable prior is, in essence, a generative model: draw a vector from a Gaussian, run it through the network, and a new sample falls out the other side. Everything that followed in the field, from diffusion models to large image and video generators, builds on the same trick of learning a mapping from a simple prior to a complex data distribution.

Examples & Exercises

THE EXERCISES START WITH A FEW NUMERICAL PASSES to fix the mechanics, then a set of concept and diagnostic prompts that target the key ideas of the chapter one at a time, and finally a hands-on notebook on MNIST. Worked solutions for the compute exercises are in the body; concept exercises carry the answer in a margin note. Try it yourself first, then check.

VAE LOSS BY THE NUMBERS. For a 1D VAE with encoder output μ=1, σ=1 on input x=3 with reconstruction $\hat{x}$=2.7, compute the total loss $\mathcal{L} = \mathcal{L}_R + \mathcal{L}_{\mathrm{KL}}$ using squared error and the Gaussian KL closed form $\frac{1}{2}(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$.

RECONSTRUCTION LOSS.

$$\begin{aligned}\mathcal{L}_R &= (x - \hat{x})^2 \\ &= (3 - 2.7)^2 \\ &= 0.3^2 \\ &= 0.09\end{aligned}$$

KL TERM.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}} &= \tfrac{1}{2}(\mu^2 + \sigma^2 - \log\sigma^2 - 1) \\ &= \tfrac{1}{2}(1 + 1 - \log 1 - 1) \\ &= \tfrac{1}{2}(1 + 1 - 0 - 1) \\ &= \tfrac{1}{2}(1) \\ &= 0.50\end{aligned}$$

TOTAL LOSS.

$$\begin{aligned}\mathcal{L} &= \mathcal{L}_R + \mathcal{L}_{\mathrm{KL}} \\ &= 0.09 + 0.50 \\ &= 0.59\end{aligned}$$

SECOND CONFIGURATION: μ=0, σ=1, x=3, $\hat{x}$=2.0.

$$\begin{aligned}
\mathcal{L}_R &= (3-2.0)^2 \\
&= 1.00 \\
\mathcal{L}_{\mathrm{KL}} &= \tfrac{1}{2}(0+1-0-1) \\
&= 0 \\
\mathcal{L} &= 1.00 + 0 \\
&= 1.00
\end{aligned}$$

The encoder matches the prior exactly so the KL drops to zero, but reconstruction more than triples. The first configuration achieves a lower total loss by spending some KL for a much better fit; that trade-off is what training resolves.

WHERE THE TRADE-OFF LIVES. Reconstruction is small in configuration one but the KL penalty is five times larger; in configuration two the KL is free but reconstruction dominates. Training finds the encoder that minimises the sum, not either term in isolation.

KL DIVERGENCE BY HAND. Compute $\mathcal{L}_{\mathrm{KL}}\big(\mathcal{N}(\mu,\sigma^2)\,\|\,\mathcal{N}(0,1)\big) = \frac{1}{2}(\mu^2+\sigma^2-\log\sigma^2-1)$ for three settings, and split each result into a shift cost $\frac{1}{2}\mu^2$ and a spread cost $\frac{1}{2}(\sigma^2-\log\sigma^2-1)$.

SETTING (I): μ=0, σ=1 (posterior matches the prior).

$$\begin{aligned}
\tfrac{1}{2}\mu^2 &= 0 \\
\tfrac{1}{2}(\sigma^2-\log\sigma^2-1) &= \tfrac{1}{2}(1-0-1) \\
&= 0 \\
\mathcal{L}_{\mathrm{KL}} &= 0+0 \\
&= 0
\end{aligned}$$

SETTING (II): μ=1, σ=1 (mean shifted, same spread).

$$\begin{aligned}
\tfrac{1}{2}\mu^2 &= \tfrac{1}{2}\times 1 \\
&= 0.50 \\
\tfrac{1}{2}(\sigma^2-\log\sigma^2-1) &= \tfrac{1}{2}(1-0-1) \\
&= 0 \\
\mathcal{L}_{\mathrm{KL}} &= 0.50+0 \\
&= 0.50
\end{aligned}$$

SETTING (III): μ=0, σ=2 (correct mean, too wide).

$$\begin{aligned}
\tfrac{1}{2}\mu^2 &= 0 \\
\tfrac{1}{2}(\sigma^2 - \log\sigma^2 - 1) &= \tfrac{1}{2}(4 - \log 4 - 1) \\
&= \tfrac{1}{2}(4 - 1.386 - 1) \\
&\approx 0.81 \\
\mathcal{L}_{\mathrm{KL}} &\approx 0 + 0.81 \\
&\approx 0.81
\end{aligned}$$

WHY THE ASYMMETRY. The shift cost is quadratic in μ. The spread cost is asymmetric: it grows roughly linearly as σ exceeds 1, but the $-\log\sigma^2$ term blows up as $\sigma \to 0$, so narrow posteriors are the most expensive of all. Setting (iii) costs more than (ii) because spreading mass away from the prior is harder than translating it.

β-VAE BALANCE PASS. Reuse the first configuration from *VAE loss by the numbers* ($\mathcal{L}_R$=0.09, $\mathcal{L}_{\mathrm{KL}}$=0.50) and compute the β-VAE loss $\mathcal{L}_\beta = \mathcal{L}_R + \beta \cdot \mathcal{L}_{\mathrm{KL}}$ for $\beta \in \{0.5, 1, 4\}$.

β=0.5.

$$\begin{aligned}
\mathcal{L}_{\beta=0.5} &= 0.09 + 0.5 \times 0.50 \\
&= 0.09 + 0.25 \\
&= 0.34
\end{aligned}$$

β=1 (STANDARD ELBO).

$$\begin{aligned}
\mathcal{L}_{\beta=1} &= 0.09 + 1 \times 0.50 \\
&= 0.09 + 0.50 \\
&= 0.59
\end{aligned}$$

β=4.

$$\begin{aligned}
\mathcal{L}_{\beta=4} &= 0.09 + 4 \times 0.50 \\
&= 0.09 + 2.00 \\
&= 2.09
\end{aligned}$$

The KL contribution to the total grows from 73% (β=0.5) to 96% (β=4). At β=4 the encoder has every incentive to shrink μ toward zero and σ toward one, at the cost of reconstruction. At β=0.5 the prior pull is relaxed and the encoder can spread out for sharper reconstructions. β=1 is the standard ELBO balance.

WHY β MATTERS. Training does not just evaluate this loss once; it follows the gradient. Whichever term dominates the loss dominates the gradient, and that is what shapes the final encoder.

REPARAMETERISATION. For μ=0.5, σ=1.2, and a sampled $\epsilon = -0.3$, compute the reparameterised sample $z = \mu + \sigma\epsilon$ and verify the gradients with respect to μ and σ.

FORWARD PASS.

$$\begin{aligned} z &= \mu + \sigma \cdot \epsilon \\ &= 0.5 + 1.2 \times (-0.3) \\ &= 0.5 - 0.36 \\ &= 0.14 \end{aligned}$$

GRADIENTS.

$$\begin{aligned} \frac{\partial z}{\partial \mu} &= \frac{\partial}{\partial \mu}(\mu + \sigma\epsilon) \\ &= 1 \\ \frac{\partial z}{\partial \sigma} &= \frac{\partial}{\partial \sigma}(\mu + \sigma\epsilon) \\ &= \epsilon \\ &= -0.3 \end{aligned}$$

Both gradients are finite scalars, so μ and σ get a clean signal from backpropagation; the randomness lives in ϵ, off the gradient path.

KL COST, BY INSPECTION. Without computing, rank the following encoder distributions by KL cost against the prior $\mathcal{N}(0, 1)$, lowest first.

- (i) μ=0, σ=1.
- (ii) μ=0, σ=0.3.
- (iii) μ=1.5, σ=1.
- (iv) μ=0, σ=3.

ANSWER. (i) < (iii) < (iv) < (ii). (i) matches the prior exactly so the KL is zero. (iii) only shifts the mean; the cost grows quadratically with μ. (iv) is too wide but the spread cost is roughly linear in σ^2. (ii) is too narrow, and the $-\log\sigma^2$ term blows up as $\sigma \to 0$, making narrow posteriors the most expensive.

AE VERSUS VAE: PREDICT THE EMBEDDING. You train a vanilla autoencoder and a VAE with k=2 on MNIST, both to convergence, and plot the validation embeddings.

- Which space concentrates encodings near the origin, and which spreads them arbitrarily across the plane?
- Does proximity in the latent space imply the inputs are visually similar? Is the answer the same for both models?

- If you pick a random point from a sparsely populated region and decode it, in which space is the result a plausible digit?
- Why k=2 and not k=10, given that MNIST has ten digit classes?

ANSWER. The AE places encodings wherever reconstruction loss is lowest; the layout is arbitrary, with no global structure and large unused regions. The VAE pulls every encoding toward the prior, so the cloud concentrates near the origin and fills the unit disc. Proximity reflects visual similarity in both, because the decoder is smooth, but only the VAE gives geometric position a global meaning: any z sampled from the prior decodes to something plausible, while a random point in an AE's empty region decodes to noise. k=2 is for visualisation; you plot $z^{(1)}$ against $z^{(2)}$ directly. At k=10 you would need t-SNE or UMAP to see the layout, and the differences you observe would be artefacts of the projection rather than of the AE-vs-VAE distinction.

PER-DIMENSION KL. For a k=4 VAE on a single input, the encoder outputs the parameters below (independent dimensions, prior $\mathcal{N}(0,1)$ per dim). Compute $\mathcal{L}_{\mathrm{KL}}^{(j)} = \frac{1}{2}(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$ for each dimension and the total $\mathcal{L}_{\mathrm{KL}} = \sum_j \mathcal{L}_{\mathrm{KL}}^{(j)}$, then label each dimension active or collapsed.

DIM 1: μ_1=1.5, σ_1=0.5.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(1)} &= \tfrac{1}{2}(1.5^2 + 0.5^2 - \log 0.5^2 - 1)\\ &= \tfrac{1}{2}(2.25 + 0.25 + 1.386 - 1)\\ &= \tfrac{1}{2}(2.886)\\ &\approx 1.44\end{aligned}$$

DIM 2: μ_2=0, σ_2=1.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(2)} &= \tfrac{1}{2}(0 + 1 - 0 - 1)\\ &= 0\end{aligned}$$

DIM 3: μ_3=0.2, σ_3=0.9.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(3)} &= \tfrac{1}{2}(0.2^2 + 0.9^2 - \log 0.9^2 - 1)\\ &= \tfrac{1}{2}(0.04 + 0.81 + 0.211 - 1)\\ &= \tfrac{1}{2}(0.061)\\ &\approx 0.03\end{aligned}$$

DIM 4: $\mu_4 = -0.8$, σ_4=0.7.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}}^{(4)} &= \tfrac{1}{2}((-0.8)^2 + 0.7^2 - \log 0.7^2 - 1)\\ &= \tfrac{1}{2}(0.64 + 0.49 + 0.713 - 1)\\ &= \tfrac{1}{2}(0.843)\\ &\approx 0.42\end{aligned}$$

TOTAL AND VERDICT.

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}} &= 1.44 + 0 + 0.03 + 0.42\\ &\approx 1.89\end{aligned}$$

Dimensions 1 and 4 are active: large per-dim KL, σ well below 1, mean clearly away from zero. Dimension 2 has collapsed to the prior. Dimension 3 is borderline: nearly at the prior, contributing almost no information.

WHAT THE PER-DIM KL TELLS YOU. Posterior collapse is a per-dimension phenomenon: a single VAE can have some dimensions active and others collapsed in the same training run. Summing the KL hides that detail; reading it per dimension exposes which axes the encoder uses and which it has given up on.

COUNTING ACTIVE UNITS. A VAE with k=8 reports the following per-dimension posterior statistics, averaged over the validation set: the squared mean $\overline{\mu_j^2}$, the variance σ_j^2, and the resulting per-dimension KL $\mathcal{L}_{\mathrm{KL}}^{(j)} = \frac{1}{2}(\overline{\mu_j^2} + \sigma_j^2 - \log \sigma_j^2 - 1)$.

Dim j	1	2	3	4	5	6	7	8
$\overline{\mu_j^2}$	1.50	0.01	0.80	0.00	1.20	0.00	0.60	0.70
σ_j^2	0.12	0.97	0.31	0.99	0.18	1.00	0.41	0.95
$\mathcal{L}_{\mathrm{KL}}^{(j)}$	1.37	0.01	0.64	0.00	1.05	0.00	0.45	0.35

Table 1: Per-dimension posterior statistics for a k=8 VAE on the validation set. Inactive dimensions have $\overline{\mu_j^2} \approx 0$ and $\sigma_j^2 \approx 1$, so their KL contribution falls to zero and they have collapsed back to the prior. Active dimensions either spread the means (large $\overline{\mu_j^2}$), tighten the variance (small σ_j^2), or both, and pay a positive KL for the information they carry.

How many dimensions are doing real work? Which ones can you remove without losing reconstruction quality? If you re-trained at k=4, would you expect the same number of active dimensions?

ANSWER. Five active dimensions: 1, 3, 5, 7, 8, identifiable by $\mathcal{L}_{\mathrm{KL}}^{(j)}$ well above zero. Dimensions 2, 4, 6 have $\mathcal{L}_{\mathrm{KL}}^{(j)} \approx 0$ and have collapsed to the prior; dropping them costs nothing. Note that dim 8 looks borderline by variance alone ($\sigma_8^2 = 0.95$), but its mean spread $\overline{\mu_8^2} = 0.70$ keeps it carrying information, which is why the KL is the cleaner diagnostic. Re-training at k=4 should still yield about four to five active units, because the active count is set by the data's information content, not by k once k is past the elbow.

DIAGNOSE THE TRAINING RUN. Three runs of the same VAE on the same data give different end-of-training behaviour. For each, name what is happening and what you would change.

Run	$\mathcal{L}_R$	$\mathcal{L}_{\mathrm{KL}}$	**Reconstructions**
A	high	≈ 0	blurry mean digit
B	low	high	sharp but unrelated to input
C	low	moderate	sharp and semantically sound

Table 2: Three end-of-training profiles for the same VAE on the same data. One is healthy training, one is posterior collapse, and one is a memorising encoder that bypasses the prior.

ANSWER. **A** is posterior collapse: KL near zero means the encoder ignores x, so the decoder outputs the data mean. Lower β or anneal it from zero. **B** has the encoder bypassing the prior to memorise inputs (high KL, low reconstruction, but sharp outputs that do not generalise to sampled z). Raise β or check for an over-parameterised encoder. **C** is what you want: both terms contribute and reconstructions track the inputs.

KL ANNEALING BY THE NUMBERS. A linear-ramp schedule sets

$$\beta(t) = \min\left(\tfrac{t}{10}, 1\right)$$

so β rises from 0 at epoch 0 to 1 at epoch 10 and stays at 1 afterwards. Assume that on a fixed validation batch the encoder reports $\mathcal{L}_R$=0.4 and $\mathcal{L}_{\mathrm{KL}}$=1.2 throughout training (the values themselves do not change, only the weight). Compute the total loss $\mathcal{L}_\beta(t) = \mathcal{L}_R + \beta(t) \cdot \mathcal{L}_{\mathrm{KL}}$ at epochs $t \in \{1, 5, 10, 20\}$.

EPOCH 1: β=0.1.

$$\begin{aligned}\mathcal{L}_\beta(1) &= 0.4 + 0.1 \times 1.2 \\ &= 0.4 + 0.12 \\ &= 0.52\end{aligned}$$

EPOCH 5: β=0.5.

$$\begin{aligned}\mathcal{L}_\beta(5) &= 0.4 + 0.5 \times 1.2 \\ &= 0.4 + 0.60 \\ &= 1.00\end{aligned}$$

EPOCH 10: β=1.

$$\begin{aligned}\mathcal{L}_\beta(10) &= 0.4 + 1 \times 1.2 \\ &= 1.60\end{aligned}$$

EPOCH 20: β=1 (CLAMPED).

$$\mathcal{L}_\beta(20) = 1.60$$

At which epoch does the KL contribution first match the reconstruction term, and what would $\mathcal{L}(t)$ look like if you set β=1 from the start?

WHEN THE TERMS MATCH. $\beta(t) \cdot \mathcal{L}_{\mathrm{KL}} = \mathcal{L}_R$ when $\beta(t) = 0.4/1.2 \approx 0.33$, i.e. around epoch 3 on this schedule. Without the ramp, the KL would dominate from epoch 1, giving the encoder every incentive to ignore x before it has learned a useful representation; the warm-up window buys the encoder time to first encode information, then take the regularisation pressure.

READ THE OBSERVATIONS FROM THE EMBEDDING SPACE EXPLORATION. You linearly interpolate between two encoded digits in latent space and decode each step. For each described path, name what the latent space tells you.

- Path 1: a smooth morph from a 3 into an 8, passing through plausible intermediate shapes.
- Path 2: the 3 stays fixed for half the path, then suddenly jumps to the 8.
- Path 3: the path passes through several unrelated digit shapes (a 2, then a 5) before reaching the 8.

ANSWER. Path 1 indicates a well-organised latent space: nearby points decode to similar digits and the regulariser has filled the region between the two encodings. Path 2 means the encoder placed the two digits far apart with empty space in between; the prior has not pulled them close enough. Path 3 means the line crosses other class regions; that is geometrically honest but suggests the chosen direction is not a meaningful axis.

AN MNIST SUBSET TO EXPLORE. You are given the same thousand handwritten digits as in the previous chapter. Implement three pieces from scratch in numpy: a linear autoencoder, the closed-form KL divergence to a standard normal prior, and the reparameterisation trick $z = \mu + \sigma \odot \varepsilon$. Sweep k on the linear AE and inspect the reconstruction curve, then compare precomputed AE and VAE embeddings at k=2 and at $k \in \{2, 8, 32\}$. Optionally lift a k=16 VAE: bar-chart its per-dimension KL using your own implementation, and decode an interpolation between an encoded 0 and an encoded 1.

CODE is provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

1 000 MNIST digits
784 pixels each
ten classes

implement: linear AE, KL divergence, reparameterisation trick

WHAT TO OBSERVE. The linear AE elbow matches the PCA reconstruction curve, confirming that a tied-weight linear AE is PCA in disguise. The AE and VAE scatters at k=2 differ in geometry: AE encodings spread arbitrarily with large empty regions, VAE encodings concentrate near the origin and fill the unit disc. Active units saturate well below the chosen k. The interpolation passes through plausible intermediate digits, showing that the latent space is genuinely continuous.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What does the autoencoder bottleneck force the encoder to discover, and how does it relate to the PCA subspace?
- Why does a deterministic autoencoder fail as a generative model?
- In the Gaussian KL, why is a narrow posterior more expensive than a shifted or widened one?
- When would you reach for β-VAE, VQ-VAE, or a conditional VAE over the plain VAE?
- Why does the reparameterisation trick work, and what does it exploit about the Gaussian?
- What is posterior collapse, and which design lever brings it on most often?
- How does the active-unit count tell you whether k is set correctly?
- Why does sampling $z \sim \mathcal{N}(0, I)$ and decoding produce plausible new data in a VAE but not in a plain autoencoder?

RECAP of Key Concepts:

- Autoencoders learn a nonlinear θ_E, θ_D through a bottleneck; linear AE recovers PCA, nonlinear goes beyond it.
- VAEs replace the point embedding with $q(z|x)$ regularised toward a prior, giving a continuous, generative latent space.
- The Gaussian KL trades a quadratic shift cost against an asymmetric spread cost; narrow posteriors cost the most.
- β-VAE buys disentanglement, VQ-VAE buys discrete codes, conditional VAE buys controlled generation.
- The reparameterisation trick $z = \mu + \sigma \odot \varepsilon$ keeps the sample differentiable in μ and σ.
- Active-unit count diagnoses bottleneck use; KL warm-up is the standard fix for posterior collapse.
- Sampling $z \sim p(z)$ and decoding generates new data; smooth interpolation tests latent continuity.

THE ROOT OF MODEL BLINDNESS. Any model can tell you what it has learned, but not how much to trust what it says about inputs it has never seen.

UNSUPERVISED DEEP LEARNING is at root the project of modelling what counts as familiar, and the VAE does this directly: low reconstruction error and high prior likelihood under $p(z)$ mean an input resembles the training data. Yet every model trained on finite data, generative or not, has a blind spot for inputs outside its experience: a VAE's error rises on unfamiliar inputs and a classifier's softmax stays peaked on adversarial ones, but neither flags the gap in any principled way.

UNCERTAINTY ESTIMATION turns familiarity into a principled tool, and it is itself an unsupervised problem: knowing where a model's knowledge ends depends on the learned distribution of inputs, not on a held-out label. The next chapter develops model-agnostic methods for separating irreducible noise from reducible ignorance, and uses them for anomaly detection, out-of-distribution flagging, and graceful degradation in deployment.

TEASER. How does a model trained without labels know when a new input falls outside the structure it has learned?

FEEDBACK.

Uncertainty Estimation

Knowing what you don't know.

The Why

In the previous chapter we built a latent space whose density tells us where the training data lives. A VAE assigns high likelihood to inputs near the embeddings it has seen, and low likelihood to those far from them. That density is already an unsupervised signal of familiarity. What it does not give us is a principled way to act on the gap between familiar and unfamiliar, nor a way to separate genuinely ambiguous inputs from inputs the model has simply never seen.

Confidence without calibration is the fastest route to silent failure.

Uncertainty estimation closes the gap. It asks: given a model trained on finite data under specific assumptions, how reliable are its outputs for a particular input? The answer matters wherever consequences are real, including medical diagnosis, autonomous driving, fraud detection, and manufacturing quality control. A model that cannot express doubt will eventually be confidently wrong, and no downstream safeguard can compensate for an uncertainty signal that was never produced.

In order to move from confident predictions to calibrated ones we have good reason to find ways to,

- separate irreducible data noise from reducible model ignorance, so that each source of uncertainty triggers the right response.
- quantify how much a model's predictions vary across plausible parameter settings, turning a single output into a distribution over outputs.
- act on uncertainty estimates in practice, routing unreliable predictions to human reviewers and detecting inputs the model was never trained to handle.

The supervisory signal comes from the data itself: the model's own variance across forward passes, the density of training examples nearby, the reconstruction loss of an autoencoder. No labels are needed to know that an input is unfamiliar.

Hands-On Experience

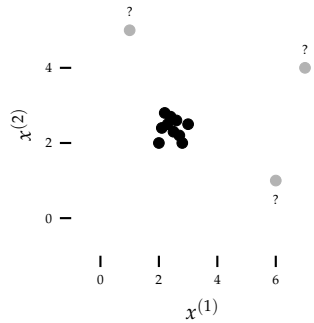


Figure 10: A cluster of training points and three grey candidates. How confident should a model be about each? What would make you trust or distrust its judgment?

GIVEN a cluster of training points and three grey candidates, ask yourself a few questions before reading on. How confident should a model be about each one? Which of the three is hardest to call, and is the difficulty due to noise in the data or to gaps in the model's experience? At what point would you stop trusting the model's judgment and pass the decision to a human?

FORM GROUPS OF FIVE. Each member is one network in an ensemble. The group will play four short games that turn the chapter's main ideas into a physical experiment.

MATERIALS. A pen and a small slip of paper for each member, plus one shared sheet for the group's tally.

STEP 1: TRAIN. Agree on a one-sentence rule that maps a number to a binary label, for example "hot if greater than the threshold, cold otherwise". Each member privately picks their own threshold uniformly between 4 and 6 and writes it on a slip of paper. The slips are hidden from the rest of the group; the small disagreement between members is what makes you an ensemble of similar but not identical models.

STEP 2: IN-DISTRIBUTION PREDICTIONS. The instructor calls out a number well inside the range, say 3 or 7. Every member shouts their prediction at the same moment. Tally agreement on the shared sheet, then repeat for two more in-distribution numbers. The ensemble converges; this is the regime where its mean prediction is reliable.

STEP 3: OUT-OF-DISTRIBUTION PREDICTIONS. Now the instructor calls a number far from anything the group trained on, say 100 or -50. Every member must still answer, but pay attention to how they answer: hesitation, half jokes, contradictory shouts. Tally agreement again. The variance has jumped, even though no member has more or less training than before. This jump is epistemic uncertainty, and it is the actionable kind: it would shrink if the group were allowed to extend its training range.

- Which inputs were unanimous, and why?
- For the OOD inputs, did anyone pick a confident wrong answer? What would have stopped them?

STEP 4: ALEATORIC INPUT. The instructor calls out a number sitting on the boundary, 5.0 when the allowed thresholds were uniform on $[4, 6]$. Members will split, but extending the training range would not help; this disagreement is irreducible noise on the input itself. It is aleatoric, and the right response is not "collect more data" but "communicate the ambiguity downstream".

STEP 5: MONTE CARLO DROPOUT. Pick one member to be the "in-

put" and the other four to be a single network. For each round, randomly close the eyes of two of the four; only the open-eyed members vote. Run five rounds with different random closures and record the votes. The variance across rounds is the MC Dropout uncertainty. Familiar inputs leave the rounds in agreement; unusual ones scatter them.

One model, many sub-models. Step 5 turns a single network into an ensemble at test time by sampling sub-networks; Steps 2 to 4 used a real ensemble of independently trained members. Both estimate epistemic uncertainty by sampling models rather than data.

Step 6: Threshold for action. On the shared sheet, plot the agreement level against each input from Steps 2 to 5. Where would you place a threshold above which a prediction is shipped automatically and below which it is routed to a human reviewer? Now imagine the human reviewer charges fifty euros per case; does the threshold change? What if a wrong shipped answer would shut down a production line for an hour?

The fundamental question is whether a model can know that it does not know. A confident wrong answer is a quiet failure; a flagged unknown is a recoverable one. Everything in this chapter, from variance across dropout masks to reconstruction error from a VAE, is one way of turning the boundary of the training distribution into a usable signal.

The Learning Objectives of this lecture:

- Distinguish aleatoric from epistemic uncertainty and explain why only the latter is reducible with more data.
- Apply MC Dropout and Deep Ensembles to quantify prediction uncertainty and interpret disagreement as model ignorance.
- Use uncertainty estimates to detect anomalies and out-of-distribution inputs, and design graceful degradation strategies that route uncertain predictions to human reviewers.

Sources of Uncertainty

Not all uncertainty is the same, and the distinction determines what we can do about it.

Two kinds of uncertainty arise in any model. Aleatoric uncertainty is inherent noise in the data and cannot be reduced by collecting more of it. A sensor with finite precision will always introduce measurement noise; overlapping class boundaries in the data will always produce ambiguous inputs. No model, however powerful, can eliminate aleatoric uncertainty.

Epistemic uncertainty reflects the model's ignorance and shrinks as more data arrives. A model trained on summer temperatures has high epistemic uncertainty about winter; show it winter data and the uncertainty decreases. Epistemic uncertainty is the actionable kind; it tells us where the model needs more evidence.

A coffee analogy. A trained barista cannot tell two identical-looking espressos apart by sight: that ambiguity is aleatoric, baked into the inputs. Show the same barista an unfamiliar single-origin and the hesitation is epistemic, an experience gap that more cups would close.

The practical consequence is that these two sources call for different responses. High aleatoric uncertainty means the problem is inherently hard at that point; the best strategy is to acknowledge the ambiguity and communicate it downstream. High epistemic uncertainty means the model has not seen enough relevant data; the best strategy is to collect more data, or to flag the prediction as unreliable.

The total predictive uncertainty decomposes into the two contributions:

$$\underbrace{\mathbb{V}\left[y \mid x\right]}_{\text{total}} = \underbrace{\mathbb{E}_\theta\big[\mathbb{V}[y \mid x, \theta]\big]}_{\text{aleatoric}} + \underbrace{\mathbb{V}_\theta\big[\mathbb{E}[y \mid x, \theta]\big]}_{\text{epistemic}}$$

The aleatoric term is the average noise level the model expects across plausible parameter settings θ; the epistemic term is the disagreement among those settings about the mean prediction. The two methods that follow, MC Dropout and deep ensembles, both estimate the second term by sampling from a distribution over models.

Quantifying Epistemic Uncertainty

Knowing what you do not know is essential for safe deployment.

Monte Carlo Dropout[13] estimates epistemic uncertainty by running T forward passes with dropout enabled at test time. In standard inference dropout is switched off; here we keep it on, so each pass uses a different randomly thinned sub-network.

[13] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1050–1059, 2016. https://arxiv.org/abs/1506.02142

Algorithm 1: MC Dropout for Uncertainty

Require: Input x, model with dropout, T forward passes
1: **for** $t = 1$ to T **do**
2: $\hat{y}_t \leftarrow$ forward pass with dropout enabled
3: **end for**
4: Mean prediction: $\bar{y} = \frac{1}{T}\sum_t \hat{y}_t$
5: Uncertainty: $\mathrm{Var}(\hat{y}) = \frac{1}{T}\sum_t (\hat{y}_t - \bar{y})^2$

High variance across passes indicates that the model is unsure about this input. Each dropout mask produces a different subnetwork, so the collection of forward passes approximates a posterior over model weights. Inputs well-represented in the training data will produce consistent predictions across masks; inputs far from the training distribution will produce scattered, disagreeing predictions.

```
def mc_dropout(model, x, T=30):
    model.eval()                          # freeze BN
    for m in model.modules():
        if isinstance(m, nn.Dropout):
            m.train()                     # keep dropout on
    preds = torch.stack([model(x) for _ in range(T)])
    return preds.mean(0), preds.var(0)
```

Practical detail. In PyTorch, `model.eval()` disables dropout. To run MC Dropout you instead leave the dropout layers in training mode while still freezing batch-norm statistics.

Deep ensembles[14] take the idea further by training M independent models and aggregating their predictions:

$$p(y|x) = \frac{1}{M}\sum_{m=1}^{M} p_m(y|x)$$

[14] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. https://arxiv.org/abs/1612.01474

Disagreement among ensemble members signals uncertainty. Unlike MC Dropout, each ensemble member is initialised and trained independently, so the diversity of predictions is driven by genuine differences in the learned representations rather than by random masking. The price is M times the training cost; the payoff is sharper, better-calibrated uncertainty, with M=5 to 10 usually enough in practice.

Application: Anomaly Detection

Anomaly detection[15] asks whether individual observations deviate from a learned notion of normal. The methods in this section can be understood as different ways of measuring how uncertain the model is about an input, through distributional assumptions, through proximity in feature space, or through reconstruction fidelity.

One direct application of uncertainty estimation is anomaly detection: when a model assigns high uncertainty to an input, it is effectively saying that this does not look like what it was trained on.

[15] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009. DOI: 10.1145/1541880.1541882. https://scholar.google.com/scholar?q=Chandola+Banerjee+Kumar+2009+anomaly+detection+survey

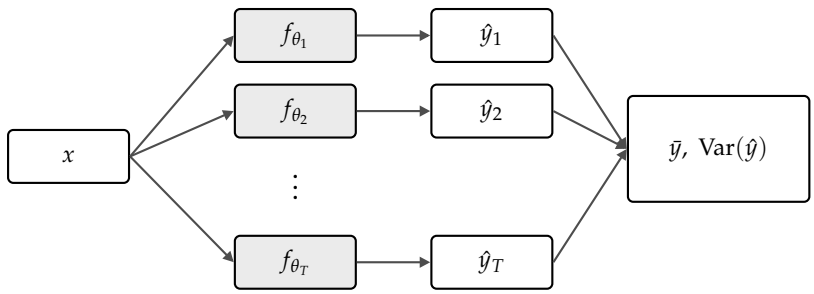


Figure 11: Sampling-based epistemic uncertainty. The same input x is passed through T samples from a distribution over models: dropout sub-networks for MC Dropout, independently trained members for a deep ensemble. The mean of the T predictions is the point estimate; their variance is the epistemic uncertainty signal.

Three kinds of anomalies appear in practice, and each calls for a different score.

Point anomalies are individual instances that are anomalous with respect to the rest of the data. A single fraudulent transaction amid millions of legitimate ones is a point anomaly.

Contextual anomalies are instances that are anomalous only within a specific context. A temperature of 30°C is normal in summer but anomalous in winter; the value itself is unremarkable, but the context makes it suspicious.

Collective anomalies are collections of instances that are individually normal but together form an anomalous pattern. A sequence of login attempts from 100 different countries within one hour is an example: each login is normal, but the pattern is not.

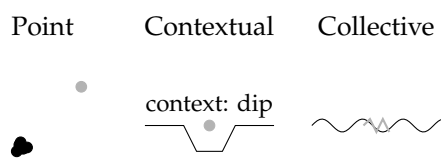


Figure 12: Three types of anomalies. Left: a point anomaly, isolated from the cluster. Centre: a contextual anomaly, the value is normal but the context, a dip, is not. Right: a collective anomaly, each point is individually normal but the oscillation pattern is unusual.

Statistical methods assume a distribution and flag low-probability points.

Parametric methods are fast and interpretable when the distributional assumption holds, brittle when it does not.

The Z-score is the simplest anomaly score for univariate data:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Flag as anomalous if $|z_i| > 3$ or another chosen threshold. The method assumes normality; heavy-tailed data will produce many false positives.

GAUSSIAN MIXTURE MODEL LIKELIHOOD extends the parametric idea to multimodal data. Fit a GMM and compute the likelihood of each observation:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

Low $p(x)$ indicates an anomaly. The number of components K must be chosen carefully; too few and normal variation is flagged, too many and anomalies are absorbed into their own component.

EXTREME VALUE THEORY handles heavy-tailed distributions by modelling the tail explicitly via the Generalised Pareto Distribution:

$$F(x) = 1 - \left(1 + \frac{\xi\, x}{\sigma}\right)^{-1/\xi}$$

PROXIMITY-BASED METHODS make precise the intuition that anomalies are far from their neighbours, with density either ignored or compensated for.

DISTANCE AND DENSITY. k-NN ranks by distance alone; LOF asks whether the local density looks like that of the surroundings.

k-NN DISTANCE assigns each point an anomaly score equal to its distance to its k-th nearest neighbour:

$$\text{score}(x) = d(x,\, x^{(k)})$$

Points far from their neighbours score high. The method is simple and effective but sensitive to the choice of k and to varying cluster densities.

LOCAL OUTLIER FACTOR[16] accounts for varying density by comparing each point's local reachability density to that of its neighbours:

$$\text{LOF}_k(x) = \frac{1}{|N_k(x)|} \sum_{y \in N_k(x)} \frac{\text{lrd}_k(y)}{\text{lrd}_k(x)}$$

where $\text{lrd}_k(x)$ is the local reachability density. LOF ≈ 1 means the point sits at a density similar to its neighbours; LOF $\gg 1$ means it sits in a sparser region and is likely anomalous.

[16] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander. LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000. DOI: 10.1145/342009.335388. https://scholar.google.com/scholar?q=Breunig+Kriegel+Ng+Sander+2000+LOF+identifying+density-based+local+outliers

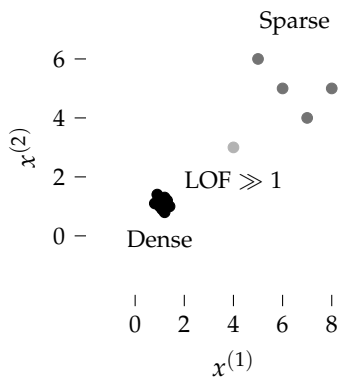


Figure 13: LOF detects the anomaly between the two clusters despite the density difference. A global threshold, as used by k-NN distance, would either miss this point or flag the sparse cluster entirely.

Reconstruction-based methods use a generative model trained on normal data and turn its reconstruction loss into an anomaly score. This is exactly the bridge from the previous chapter: a VAE that places an input far from its prior, or that reconstructs it poorly, is telling us the input does not belong.

If the model cannot reconstruct it, it has probably never seen anything like it.

Autoencoder reconstruction error provides a natural anomaly score. Train an autoencoder on normal data only, then score each test point by how poorly it reconstructs:

$$\text{score}(x) = \|x - \text{decode}(\text{encode}(x))\|^2$$

High reconstruction error indicates the model has not learned to represent this type of input.

VAE log-likelihood refines the idea by using the negative ELBO as the anomaly score:

$$\begin{aligned}\text{score}(x) &= -\text{ELBO}(x) \\ &= -\mathbb{E}_{q(z|x)}[\log p(x|z)] + D_{\text{KL}}\big(q(z|x) \,\|\, p(z)\big)\end{aligned}$$

An anomalous input will both reconstruct poorly and require an unusual latent encoding, contributing to both terms.

Application: Out-of-Distribution Detection

OOD detection formalises the question: is this input from the training distribution at all?

Where anomaly detection asks whether a point deviates from the training distribution, out-of-distribution detection frames this as a binary decision problem: in-distribution or out. The distinction matters because OOD methods are designed for the regime where

the input is so far from the training data that the model's learned representations may be entirely unreliable.

SOFTMAX CONFIDENCE is the naive baseline: flag inputs where $\max_k p(y{=}k|x)$ is low. The problem is that neural networks are often confidently wrong on OOD data; the softmax normalises to sum to one regardless of how far the input is from any training example.

ODIN[17] improves on softmax confidence by combining temperature scaling with input perturbation:

$$\tilde{x} = x - \epsilon \cdot \operatorname{sign}\big(\nabla_x \log p(y{=}\hat{y}|x;\, T)\big)$$

[17] S. Liang, Y. Li, and R. Srikant. Enhancing the reliability of out-of-distribution image detection in neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1706.02690

ENERGY-BASED OOD DETECTION[18] uses the free energy of the logits directly:

$$E(x) = -\log \sum_k \exp\big(f_k(x)\big)$$

[18] W. Liu, X. Wang, J. Owens, and Y. Li. Energy-based out-of-distribution detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. https://arxiv.org/abs/2010.03759

Higher energy indicates OOD. Unlike softmax confidence, the energy score is not normalised, so it can distinguish low-density regions from high-density ones.

Evaluation Without Labels

WITHOUT GROUND-TRUTH LABELS, internal metrics are the only quantitative feedback we have.

THE SILHOUETTE SCORE measures how similar each point is to its own cluster compared to the nearest other cluster:

$$s(i) = \frac{b(i) - a(i)}{\max\big(a(i),\, b(i)\big)}$$

Averaged over all points, it ranges from -1 to 1; higher is better.

THE CALINSKI-HARABASZ INDEX takes a different view, comparing between-cluster dispersion to within-cluster dispersion:

$$CH = \frac{SS_B/(K-1)}{SS_W/(N-K)}$$

Higher values indicate tighter, better-separated clusters.

STABILITY ANALYSIS provides a complementary signal. Run clustering multiple times with different initialisations or subsampled data,

then measure how consistently points are assigned to the same cluster. Stable assignments suggest that the structure is genuine rather than an artefact of the algorithm.

Acting Under Uncertainty: Graceful Degradation

Safe AI means knowing when to hand off to a human.

Quantifying uncertainty is only useful if we act on it. Deployed ML systems must handle the inevitable cases they cannot confidently resolve. Four strategies compose into a practical safety net.

Confidence thresholds: only act autonomously on predictions above a calibrated confidence level.

Human handoff: route uncertain or high-stakes cases to human reviewers.

Fail-safe defaults: when uncertain, take the conservative action, for example holding a transaction rather than approving it.

Monitoring: track prediction confidence and anomaly scores over time; alert on distribution shift before it causes silent failures.

Examples & Exercises

COMPUTING BY HAND builds the intuition that no anomaly score can replace. Step through the arithmetic slowly.

Z-SCORE ON A SENSOR READING. Daily mean temperatures over a week read $\{19, 21, 20, 22, 19, 20, 35\}$ °C. Compute the Z-score of the last reading and decide whether it crosses the conventional $|z|>3$ threshold.

$$\begin{aligned}\mu &= \tfrac{19+21+20+22+19+20+35}{7}\\ &= 22.3\end{aligned}$$

$$\sigma \approx 5.42$$

$$\begin{aligned}z_7 &= \tfrac{35-22.3}{5.42}\\ &\approx 2.34\end{aligned}$$

Below the conventional threshold of 3, but 35 °C is clearly outside the typical week's spread. Heavy-tailed or small-sample data routinely produces this kind of false negative; the Z-score is a starting point, not a verdict.

LOF BY HAND. Given five points $x_1{=}(0,0)$, $x_2{=}(1,0)$, $x_3{=}(0,1)$, $x_4{=}(1,1)$, $x_5{=}(5,5)$ with $k{=}2$, compute the 2-distance (the distance to the second-nearest neighbour) for each point and decide which has the highest LOF.

The four cluster points have a 2-distance of approximately 1.41 each, since the second nearest neighbour sits at the diagonal of the unit square:

$$\begin{aligned}d_2(x_1) &= d(x_1, x_4)\\ &= \sqrt{1^2+1^2}\\ &= 1.41\end{aligned}$$

For $x_5{=}(5,5)$, the nearest cluster point is $x_4{=}(1,1)$:

$$\begin{aligned}d(x_5, x_4) &= \sqrt{4^2+4^2}\\ &= 5.66\end{aligned}$$

The second nearest cluster point is either x_2 or x_3, both at distance $\sqrt{4^2+5^2} \approx 6.40$. So $d_2(x_5) \approx 6.40$, more than four times the cluster's 2-distance. The local reachability density of x_5 is correspondingly low while its neighbours' densities stay high, so $\mathrm{LOF}_2(x_5) \gg 1$ and the four cluster points sit at $\mathrm{LOF}_2 \approx 1$.

WHY k-NN DISTANCE ALSO CATCHES THIS. With one isolated outlier and a single dense cluster, simple k-NN distance is enough. LOF earns its keep when two clusters of different density coexist; see Figure 12.

RECONSTRUCTION ANOMALY. You train an autoencoder on MNIST digits 0 to 8. At test time, you present digit 9. What reconstruction error do you expect? Under what circumstances might the autoencoder still produce a low error, and what does that tell you about the limits of reconstruction-based detection?

The model never learned to encode or decode 9, so the error should be high. However, the autoencoder may reconstruct 9 as a visually similar digit such as 4 or 7, producing only moderate error. This is a failure mode of reconstruction-based methods.

MC Dropout uncertainty. A classifier with dropout produces the following softmax outputs for a single input across T=5 forward passes.

Pass	$p(\textbf{cat})$	$p(\textbf{dog})$
1	0.9	0.1
2	0.4	0.6
3	0.7	0.3
4	0.5	0.5
5	0.8	0.2

Table 3: MC Dropout softmax outputs across T=5 forward passes for a single input.

Compute the predictive mean and variance for $p(\text{cat})$. Would you trust this prediction? At what variance threshold would you route this input to a human reviewer?

$$\begin{aligned}\bar{p} &= \tfrac{0.9+0.4+0.7+0.5+0.8}{5} \\ &= 0.66\end{aligned}$$

$$\begin{aligned}\text{Var} &= \tfrac{1}{5}\sum_t (p_t - 0.66)^2 \\ &\approx 0.034\end{aligned}$$

The mean leans toward cat, but the variance is substantial and pass 2 disagrees strongly. This input sits in a region of high epistemic uncertainty, and routing it to a human reviewer would be prudent.

Picking a threshold under class imbalance. You have 980 normal points and 20 injected anomalies. An anomaly score gives the following confusion at three thresholds.

Threshold	**TP**	**FP**	**Precision**	**Recall**
low	19	200	0.087	0.95
medium	14	30	0.318	0.70
high	6	2	0.750	0.30

Table 4: Confusion at three thresholds on 980 normal and 20 anomalous points.

Which threshold would you ship for fraud screening with downstream human review? Which for autonomous shutdown of a high-throughput line? The point is that a single ROC-AUC number hides the cost asymmetry; choosing a threshold is choosing which mistake you can live with.

Cheap review favours high recall. If a human checks every flagged transaction, the low threshold catches 19/20 anomalies at the cost of 200 false alarms.

Expensive shutdown favours high precision. Stopping a production line on a false positive is costly, so the high threshold and a 0.30 recall is a defensible trade if other safeguards exist.

A sensor dataset to explore. You are given 500 sensor readings from a manufacturing line, 480 normal and 20 injected anomalies. Apply k-NN distance, LOF, and autoencoder reconstruction error. Compare the three anomaly-score distributions for normal versus anomalous points. Sweep the threshold and plot the trade-off between false positives and detection rate.

What to observe. k-NN distance works well when the anomalies

Code will be provided as a Python notebook. Use it as a starting point, break things, and observe what changes.

are globally far from normal data but struggles when a dense and a sparse cluster coexist. LOF handles the density variation. The autoencoder catches subtle anomalies that are close in distance but structurally different. No single method dominates across all anomaly types.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thinking:

- What is the difference between aleatoric and epistemic uncertainty, and which one can MC Dropout estimate?
- When would you prefer deep ensembles over MC Dropout, and what are the trade-offs?
- What distinguishes a point anomaly from a contextual one, and why does the distinction matter for method choice?
- Why does the Z-score fail on heavy-tailed or multimodal data, and what alternatives exist?
- When does k-NN distance outperform LOF, and when does it fail?
- Why might an autoencoder trained only on normal data reconstruct an anomaly with low error, and how would you mitigate this?
- Why is high softmax confidence not a reliable indicator that an input is in-distribution?
- How do you set an anomaly threshold in practice when labelled anomalies are scarce?
- Why should a deployed model include a human-handoff mechanism, and how would you decide when to trigger it?
- How does the silhouette score complement the Calinski-Harabasz index, and when might both disagree?

RECAP of Key Concepts:

- Aleatoric uncertainty is irreducible noise in the data; epistemic uncertainty reflects the model's ignorance and shrinks with more data. Only the second is actionable.
- MC Dropout approximates a posterior over weights by leaving dropout on at test time; deep ensembles do the same with M independently trained models, at a higher training cost but sharper, better-calibrated outputs.

- Anomaly detection turns uncertainty into a usable score via statistical, proximity-based, or reconstruction-based methods. The type of anomaly, point, contextual, or collective, determines which family fits.
- OOD detection asks whether an input is in the training distribution at all. Softmax confidence is unreliable on far inputs; energy and ODIN exploit the unnormalised logits to do better.
- A practical safety net stacks confidence thresholds, human handoff, fail-safe defaults, and distribution-shift monitoring. Each compensates for what the others miss.

THE SHARED LIMITATION across MC Dropout, ensembles, anomaly scores, and OOD detectors is that each defines unfamiliar with respect to the training distribution it inherits. If the training set itself contains unlabelled anomalies, or if the notion of normal drifts over time, the detector loses its reference and starts certifying as familiar what should have been flagged. The 9 that an autoencoder trained on 0 to 8 reconstructs as a 7 is the smallest version of this problem; large pretrained models on shifting web data are the largest.

THE ROOT OF MODEL BLINDNESS. Every method in this chapter inherits its notion of normal from the training data; when that notion is wrong or shifts, the detector degrades silently.

SELF-SUPERVISED LEARNING attacks the same blindness from the opposite direction. Instead of asking which inputs are unfamiliar relative to a fixed training set, it builds representations whose structure comes from the data itself, organised by augmentations, masking, or contrastive pairs rather than by labels. Those representations then feed every downstream task, including the clustering of Part I and the anomaly detection of this chapter, on a single shared backbone.

TEASER. Can the data supervise itself?

FEEDBACK

Part III

Epistemic Pattern

Self-Supervised Learning

Labels are expensive. Let data supervise itself.

The Why

Unlabeled data is abundant; labels are scarce and expensive.

In the previous chapter we treated unfamiliarity as a signal: a model knows it does not know when its predictions disagree across forward passes, when a point sits far from its neighbours, or when a VAE cannot reconstruct it. Each method inherits its notion of normal from the labelled or curated training set it was given. The bottleneck is the training set itself.

Labels are expensive. ImageNet took fourteen million images and years of human annotation. Medical imaging labels run from fifty to five hundred dollars per expert pass. A single autonomous driving scene takes hours of 3D box drawing per frame. Meanwhile unlabelled data is essentially unlimited: the web holds billions of images, trillions of text tokens, and endless video.

Self-supervised learning closes that gap by extracting supervision from the data itself. We design pretext tasks, auxiliary objectives whose labels come for free from the structure of the input: predict the rotation that was applied, the patch that was masked, the next token in the sequence. The model has to understand the data well enough to solve them, and the representations it builds along the way carry over to downstream tasks where labels are scarce.

In order to move from expensive manual annotation to learning from raw data we have good reason to find ways to,

- design auxiliary tasks whose supervision signal is inherent in the data itself.
- learn representations that capture semantic structure without any human labels.

- transfer learned features to downstream tasks where labels are scarce.

Figure 14: Self-supervised learning pipeline: design a pretext task on unlabelled data, learn representations from it, then transfer those representations to downstream tasks where labels are scarce.

Hands-On Experience

CONSIDER a single image and three pretext tasks the model could be asked to solve. For each one, sketch the input it sees, the label it is asked to predict, and what the model would have to understand about the image to succeed.

- Rotate the image by $0°, 90°, 180°, 270°$ and predict the rotation angle.
- Split the image into nine patches, shuffle them, and predict the original arrangement.
- Convert the image to grayscale and predict the original colours.

FORM GROUPS OF THREE and pick one of the three pretext tasks above. Each group also needs a printed photograph or magazine page that can be cut, rotated, and marked up; choose an image that contains a recognisable object rather than an abstract pattern.

A PRETEXT TASK should be hard enough to require understanding image structure, easy enough that the model can actually learn it.

STEP 1: APPLY THE AUGMENTATION. Perform the chosen task on your photo. Rotate it to one of the four angles, cut it into nine patches and shuffle them, or hide the colours and keep only a grayscale copy. Pass the augmented version to a neighbouring group; keep the original hidden.

SCORING. For every round, the receiving group writes down their answer and a one-sentence description of how they reached it. Correct answer = 0, wrong answer = 1. At the end, tally the strategies: how many were content based, and how many exploited a shortcut?

STEP 2: PREDICT. The receiving group has sixty seconds to recover the rotation angle, the patch order, or the original colours. They write down both their answer and the strategy they used to reach it.

STEP 3: REVEAL THE SHORTCUT. Compare answers and strategies. A page number in the corner gave away the rotation. A continuous colour gradient solved the jigsaw without parsing any object. The fact that skies are blue and grass is green made the colourisation guessable without recognising what was in the image. A pretext task is only as informative as the shortcuts it leaves available.

- Which strategy did the receiving group rely on?
- Would the same strategy work on a different image, or did it exploit something specific to this one?

STEP 4: FORECLOSE THE SHORTCUT. Modify the augmentation so the obvious shortcut is gone. Crop the corner that revealed orientation. Pick patches whose seams cut through textured regions instead of along clean edges. Choose a photo without sky or grass for the colourisation game. Pass the new version to a different group and run the prediction again.

STEP 5: TWO VIEWS. Now switch from prediction to contrast. Each group prepares two non-overlapping crops of their original image, for example a top half and a bottom half, or two random patches taken from far apart in the frame. Pool every crop in the room into a single pile. The challenge is to pair the two crops that came from the same image and to push apart crops that came from different images.

STEP 6: REFLECT. Crops from the same image share lighting, palette, and recurring textures, and a model that only matches on those low level cues will solve the easy version of the game and fail on a real test set. The crops that pair correctly when the cues are stripped away are the ones whose match relies on the actual content of the image. This is exactly the pressure that contrastive learning puts on a representation.

THE FUNDAMENTAL QUESTION, then, is not whether each task is solvable but what shortcut a lazy model could take. The rest of this chapter is about choosing pretext signals whose only winning strategy is to learn a transferable representation, and about diagnosing the failure modes that appear when the shortcut wins.

TWO PARADIGMS, ONE PRINCIPLE. Steps 1 to 4 are predictive: the supervision is what was hidden. Steps 5 and 6 are contrastive: the supervision is which views belong together. Both turn the structure of unlabelled data into a free training signal, with no annotator in the loop.

THE LEARNING OBJECTIVES of this lecture:

- Design pretext tasks and contrastive objectives that yield transferable representations.
- Distinguish contrastive methods from non-contrastive and masked-prediction approaches, and identify what prevents collapse in each.
- Apply masked prediction for both vision and language modalities.

Pretext Tasks

A pretext task creates pseudo-labels from data structure.

EARLY SELF-SUPERVISED METHODS designed clever pretext tasks by hand. Each one builds a pseudo-label out of an operation we can apply to the data and then ask the model to invert.

Task	**Supervision signal**
Rotation prediction	Predict the applied rotation angle ($0°, 90°, 180°, 270°$).
Jigsaw puzzle	Predict the permutation of shuffled patches.
Colourisation	Predict the original colours from a grayscale input.
Inpainting	Predict the pixels in a masked region.
Context prediction	Predict the relative position of two patches.

Table 5: Classic handcrafted pretext tasks for images. Each one converts a free operation on the data into a label the model can learn from.

THE LIMITATION of handcrafted pretext tasks is that their performance does not reliably correlate with downstream performance. A model can become very good at predicting rotation while learning a representation that captures little of what makes the image semantically interesting. The next sections move from clever tasks to more principled objectives.

Contrastive Learning

CORE IDEA. Pull similar things together, push different things apart.

CONTRASTIVE LEARNING replaces the handcrafted pretext task with a single, generic objective. From every sample we build a positive pair, two augmented views of the same image, and a set of negative pairs drawn from other images. The model is asked to identify which view among the candidates belongs to the anchor.

THE INFONCE LOSS formalises this as a softmax cross-entropy over the candidates. Given an anchor x, its positive view x^+, and a set of negatives $\{x_k^-\}$, all encoded into embeddings z, the loss is:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(z, z^+)/\tau)}{\exp(\text{sim}(z, z^+)/\tau) + \sum_k \exp(\text{sim}(z, z_k^-)/\tau)} \tag{1}$$

where $\text{sim}(u, v) = u \cdot v / (\|u\| \, \|v\|)$ is cosine similarity and τ is the temperature [19]. Lower τ sharpens the softmax, making the loss focus on the hardest negatives; higher τ softens it, treating all negatives more uniformly.

[19] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. https://arxiv.org/abs/1807.03748

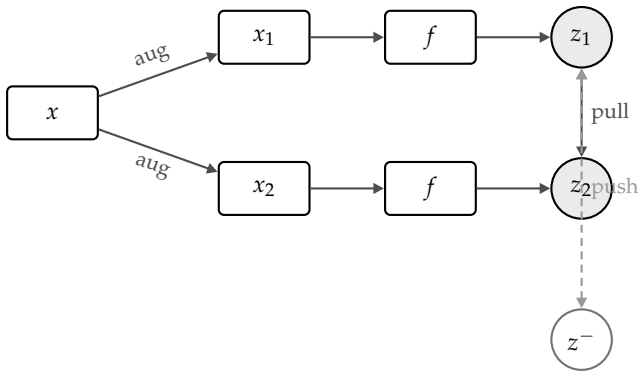


Figure 15: Contrastive learning. Two augmented views x_1, x_2 of the same input x pass through a shared encoder f and are pulled together in embedding space; an embedding z^- from a different input is pushed away.

SimCLR[20] turns this into a clean training recipe. Each minibatch of N images is doubled to $2N$ augmented views, and every other view in the batch is treated as a negative.

[20] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1597–1607, 2020. https://arxiv.org/abs/2002.05709

Algorithm 12 SimCLR

Require: Encoder f, projection head g, augmentation distribution $\mathcal{T}$, temperature τ

```
for each minibatch of N images do
    Sample two augmentations per image to get 2N views
    for each positive pair (i, j) from the same image do
        z_i ← g(f(x_i))
        z_j ← g(f(x_j))
        Treat the other 2(N − 1) views as negatives
        Backpropagate the InfoNCE loss
    end for
end for
```

Simple Contrastive Learning of Visual Representations: encoder, projection head, augmentation, batch.

Key ingredients of SimCLR:

- Strong augmentations: random crop, colour jitter, Gaussian blur. Without them, the model takes shortcuts.
- Projection head g: a small MLP placed after the encoder during training and discarded afterwards. The contrastive loss runs on the projected space; downstream tasks use the encoder output.
- Large batch size: more negatives in each minibatch sharpen the contrast, but require thousands of in-batch examples to be effective.
- Temperature $\tau \approx 0.1$ in practice; lower values overemphasise hard negatives, higher values wash the loss out.

MoCo[21] relaxes the large-batch requirement by storing negatives in a queue rather than recomputing them every step. A momentum encoder produces the keys that fill this queue, and is updated as a slow exponential moving average of the query encoder:

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q$$

with $m \approx 0.999$. The queue stores K recent key embeddings, providing a large negative set that does not need to fit in a single forward pass; MoCo trains effectively at batch sizes around 256 where SimCLR would need 4,000 or more.

[21] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9729–9738, 2020. https://arxiv.org/abs/1911.05722

MOMENTUM CONTRAST. A queue of recent keys plus a slowly updated key encoder gives consistency without huge batches.

Non-Contrastive Methods

Surprising finding: you don't need negatives!

BYOL[22], Bootstrap Your Own Latent, learns without negative pairs at all. Two networks run in parallel: an online network with encoder, projector, and a small predictor head, and a target network of the same shape but without the predictor. The target is a momentum copy of the online encoder, updated as in MoCo. The loss is the mean squared error between the online prediction and the target projection of a different augmented view:

$$\mathcal{L}_{\text{BYOL}} = \|q_\theta(z_1) - \text{sg}(z_2')\|^2$$

where sg is the stop-gradient, blocking backpropagation through the target. The asymmetry between the two branches is what keeps the system from collapsing onto a single constant embedding.

[22] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020. https://arxiv.org/abs/2006.07733

SIMSIAM[23] simplifies further by dropping the momentum encoder entirely; both branches share weights, only the predictor and the stop-gradient differ:

$$\mathcal{L}_{\text{SimSiam}} = -\tfrac{1}{2}(\text{sim}(p_1, \text{sg}(z_2)) + \text{sim}(p_2, \text{sg}(z_1)))$$

[23] X. Chen and K. He. Exploring simple siamese representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15750–15758, 2021. https://arxiv.org/abs/2011.10566

WHY DON'T THEY COLLAPSE? Without negatives, nothing in the loss obviously prevents both branches from mapping every input to the same vector. The empirical answer is that three design choices conspire to break symmetry:

- the predictor on only one branch, so the two outputs cannot be directly identified.

- the stop-gradient, which freezes one branch as a target and stops the gradient from collapsing both at once.
- batch normalisation, which centres and rescales activations within each batch and so implicitly contrasts each sample against the others.

Masked Prediction

MASK PART OF THE INPUT; predict the masked part.

MASKED PREDICTION sits across the room from contrastive learning. Instead of comparing two views of an input, the model sees a single corrupted view and is asked to recover what was hidden. The pretext task is reconstruction; the only label is the original input itself.

BERT[24] pioneered masked prediction for language. The recipe is mechanical:

- Mask 15% of input tokens uniformly at random.
- Of those, replace 80% with a special `[MASK]` token, 10% with a random token, and leave 10% unchanged. The mix prevents the model from only learning to fill in `[MASK]` and forces it to attend to context everywhere.
- Train the model to predict the original token at each masked position from the surrounding context.

[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019. https://arxiv.org/abs/1810.04805

$$\mathcal{L}_{\text{MLM}} = -\sum_{i \in \text{masked}} \log p(x_i \mid x_{\setminus i})$$

MAE[25] carries the same idea over to vision, with one structural twist. Images are split into patches, typically 16×16 pixels, and 75% of them are masked at random. Only the visible patches are passed to the transformer encoder; a lightweight decoder then receives the encoded visible tokens together with shared learnable mask tokens at the missing positions and reconstructs raw pixels. Encoding only the visible patches makes pretraining cheap; the high masking ratio forces the model to learn long-range structure rather than local interpolation.

[25] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022. https://arxiv.org/abs/2111.06377

MASKED AUTOENCODERS mask 75% of patches, encode only the visible ones, and reconstruct pixels with a small decoder.

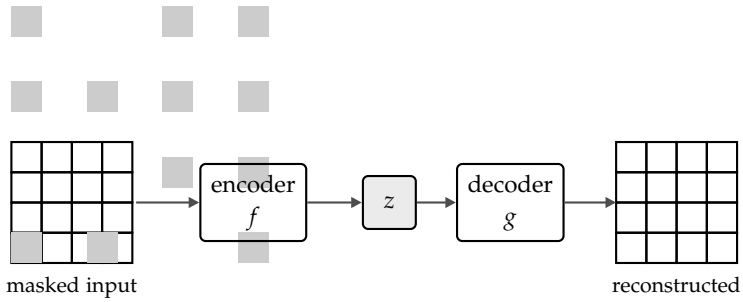


Figure 16: MAE: 75% of patches are masked, the encoder f runs only on the visible patches and produces a compact embedding z, and a small decoder g reconstructs the pixels of the full image using the encoded tokens together with shared learnable mask placeholders.

KNOWLEDGE DISTILLATION[26] appears here because the next method depends on it. A large teacher model is compressed into a smaller student by training the student to match the teacher's soft output distribution rather than hard labels.

[26] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. https://arxiv.org/abs/1503.02531

DISTILLATION transfers knowledge from a large teacher to a small student.

SOFT LABELS are produced by scaling the teacher's logits with a temperature T:

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Higher T yields a softer distribution that reveals more about the teacher's learned structure. The student trains to match these soft targets, inheriting not just the teacher's predictions but the structure of its uncertainty across classes.

DATA2VEC[27] generalises masked prediction across modalities, including vision, speech, and text, using a single framework with teacher-student distillation. Rather than predicting raw pixels, audio samples, or tokens, the student predicts the teacher's latent representations of masked inputs, applying the distillation idea above to self-supervised pretraining. The same loss therefore trains the same architecture on three different signals, all without modality-specific design choices.

[27] A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli. data2vec: A general framework for self-supervised learning in speech, vision and language. *arXiv preprint arXiv:2202.03555*, 2022. https://arxiv.org/abs/2202.03555

Cross-Modal Self-Supervision

CLIP[28] learns joint image-text representations from web data. The supervision signal is the noisy correspondence between an image and its caption: across a batch of N image-caption pairs, the diagonal of the $N \times N$ similarity matrix should score higher than the off-diagonal entries.

CONTRASTIVE LANGUAGE-IMAGE Pre-training pairs an image encoder and a text encoder so that captions and images live in the same space.

[28] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021. https://arxiv.org/abs/2103.00020

- Trained on roughly 400 million image-caption pairs scraped from the web.
- Contrastive InfoNCE loss applied symmetrically across the image and text directions.
- The result is a shared embedding space that supports zero-shot transfer: classify a new image by comparing it to the text embedding of each candidate label.

This bridges directly to the next chapter, where these joint embeddings carry classification, retrieval, and grounding to tasks the model was never explicitly trained on.

Examples & Exercises

COMPUTING BY HAND builds the intuition that no library call can replace. Step through the arithmetic slowly.

INFONCE BY HAND. A single anchor z has cosine similarities $\text{sim}(z, z^+)$=0.9 to its positive view and $\text{sim}(z, z_1^-)$=0.3, $\text{sim}(z, z_2^-)$=0.1 to two negatives. Compute the InfoNCE loss at temperature τ=0.5.

Plug into the definition and start with the scaled similarities:

$$\begin{aligned} s^+ &= \tfrac{0.9}{0.5} \\ &= 1.8 \\ s_1^- &= \tfrac{0.3}{0.5} \\ &= 0.6 \\ s_2^- &= \tfrac{0.1}{0.5} \\ &= 0.2 \end{aligned}$$

Exponentiate and sum:

$$\begin{aligned} \exp(s^+) &\approx 6.05 \\ \exp(s_1^-) &\approx 1.82 \\ \exp(s_2^-) &\approx 1.22 \\ \text{denominator} &= 6.05 + 1.82 + 1.22 \\ &= 9.09 \end{aligned}$$

The loss is the negative log of the positive's share:

$$\begin{aligned} \mathcal{L}_{\text{InfoNCE}} &= -\log \tfrac{6.05}{9.09} \\ &= -\log(0.665) \\ &\approx 0.41 \end{aligned}$$

EFFECT OF τ. If you redo the calculation at τ=0.1 the positive's share jumps to roughly 1.0 and the loss falls toward zero; at τ=1 the three candidates look more equal and the loss climbs above 0.7. Lower τ rewards the model only when the positive sits clearly above the negatives.

DESIGN A PRETEXT TASK. Propose one self-supervised pretext task for each modality and name a shortcut a lazy model could exploit if you got the design wrong.

- Audio, for example speech recordings.

- Time series, for example sensor readings.
- Graphs, for example molecular structures.

Reference answers. Audio: predict the next 200 ms from the past second; shortcut is silence padding. Time series: predict masked timestamps; shortcut is local interpolation when masking is too short. Graphs: predict masked edges; shortcut is exploiting node degree alone.

Augmentation ablation. In SimCLR, what role does random cropping play, and what would happen if you removed it but kept colour jitter and Gaussian blur? What invariance does each augmentation buy, and which one buys the most?

Why crops dominate. Random cropping forces the encoder to recognise the same object from very different views, which is the cheapest way to manufacture genuinely informative positive pairs. Without it the network learns to match colour histograms and surface texture and generalises poorly to downstream tasks.

BYOL collapse check. You disable the predictor head in BYOL and keep everything else identical. The training loss drops faster than usual, but downstream accuracy is near random. What happened, and which design lever would you reintroduce first?

Symmetry collapse. Without the predictor, the two branches become identical and the loss is minimised by mapping every input to a single point, the trivial constant solution. The predictor head is the asymmetry that prevents this; restore it before tuning anything else.

A coffee imagery dataset to explore. You are given 5,000 unlabelled coffee photos and asked to pretrain a SimCLR encoder. Implement the augmentation pipeline (random crop, colour jitter, Gaussian blur), the projection head, and the symmetric InfoNCE loss. Train for a fixed budget. Then freeze the encoder and use a linear probe on a small labelled subset of 200 images, four classes (espresso, latte, drip, cold brew), to evaluate the representation. Sweep the temperature $\tau \in \{0.05, 0.1, 0.5\}$ and the batch size $N \in \{128, 512\}$ and chart linear-probe accuracy.

Code will be provided as a Python notebook.
Use it as a starting point, break things, and observe what changes.

5,000 unlabelled coffee photos
224×224 pixels each
two augmentations per image

ResNet18 encoder, 128-dim projection, $\tau = 0.1$

What to observe. The augmentation pipeline matters most: drop random cropping and accuracy collapses to near random, even at large batch sizes. Lower τ improves accuracy until it underflows, at which point the loss saturates and training stalls. Larger batches give a clean, monotonic improvement that flattens after a few hundred negatives, demonstrating why MoCo's queue is worth the engineering for small-batch settings.

Self-Reflection and Recap

Self-Reflection questions to guide your thinking:

- Why is the projection head discarded after training in SimCLR, and what does that say about where the useful representation actually lives?

- What prevents collapse in non-contrastive methods like BYOL and SimSiam, and which design lever fails first if you remove it?
- How does the masking ratio in MAE affect what the model must learn, and why does 75% work better in vision than 15% does in language?
- When would you reach for contrastive learning over masked prediction, and when the other way around?
- How does the choice of augmentation define what the representation becomes invariant to, and what shortcuts open up if augmentations are too weak?
- What failure modes arise when the pretext task is too easy, and what happens when it is too hard?
- Why does cross-modal pretraining as in CLIP enable zero-shot transfer when single-modality pretraining does not?
- How does the unsupervised story so far, similarity, clustering, dimensionality reduction, generative latent spaces, uncertainty, self-supervision, hold together as a single thread?

RECAP of Key Concepts:

- Self-supervision turns the structure of unlabelled data into a learning signal. Pretext tasks were the first attempt; contrastive, non-contrastive, and masked-prediction objectives now dominate.
- Contrastive methods optimise InfoNCE: pull augmented views of the same input together, push views from different inputs apart. SimCLR pays for negatives in batch size; MoCo pays for them with a queue and a momentum encoder.
- Non-contrastive methods (BYOL, SimSiam) avoid negatives entirely. Asymmetry, stop-gradient, and batch normalisation conspire to prevent collapse, with no single mechanism the whole story.
- Masked prediction reconstructs hidden tokens (BERT) or hidden patches (MAE), forcing the model to capture long-range dependencies; Data2Vec unifies the recipe across vision, speech, and text using teacher-student distillation.
- Augmentation choices define the invariances of the learned representation; cropping is the workhorse in vision and the only augmentation whose removal collapses downstream accuracy on its own.

- CLIP turns the noisy correspondence between web images and captions into a shared embedding space, and that joint space is the foundation for zero-shot transfer.

THE SHARED LIMITATION of every method in this chapter is that learning the representation and using the representation are still two different problems. SimCLR and MAE produce a backbone whose embeddings are rich, geometrically organised, and transferable; what they do not provide is a recipe for adapting that backbone to a downstream class the encoder was not exposed to. Pretraining ends at the backbone; the gap to a usable classifier remains.

THE ROOT OF REPRESENTATION HUNGER. Every method in this chapter delivers features without labels, but no method tells you how to use those features when the downstream task contains a class or a domain the encoder has never seen.

ZERO-SHOT AND FEW-SHOT LEARNING close that gap. They take the shared embedding space that CLIP, MAE, or any other self-supervised model leaves behind and turn it into a classifier on the fly: in zero-shot by writing the class as a text prompt and embedding it alongside the image, in few-shot by anchoring the prediction in a handful of labelled examples. The next chapter develops both directions.

TEASER. Can the same backbone classify a class it has never been trained on?

FEEDBACK

Index

aleatoric uncertainty, 39, 42
Anomaly Detection, 39, 43
autoencoder, 15, 20, 22

BERT, 55, 61
β-VAE, 23
bottleneck design, 25
BYOL, 60

Calinski-Harabasz index, 47
CLIP, 62
collective anomaly, 44
conditional VAE, 23
contextual anomaly, 44
contrastive learning, 58

Data2Vec, 62
deep ensembles, 42, 43
deployed ML, 39
dimensionality reduction, 21
dropout, 42

ELBO, 15, 22
embedding, 15
energy score, 47
epistemic uncertainty, 39, 42
evidence lower bound, 22
examples, 28, 49, 64
exercises, 28, 49, 64
Extreme Value Theory, 45

generation, 27
GMM
  anomaly detection, 44
graceful degradation, 47
gradient descent, 20

human handoff, 47

InfoNCE, 58

k-NN distance, 45
KL divergence, 22
knowledge distillation, 62

latent dimensionality, 25
latent interpolation, 27
latent variable model, 15
license, 2
Local Outlier Factor, 45

MAE, 61
manifold, 15
masked autoencoder, 61
masked language modeling, 61
masked prediction, 61
MoCo, 60
momentum encoder, 60
Monte Carlo Dropout, 42

non-contrastive learning, 60

ODIN, 46
OOD Detection, 39
OOD detection, 46
outlier detection, 43

PCA, 15
point anomaly, 44
pretext task, 55, 58

recap, 36, 51, 65
reconstruction error, 20, 45
reflection, 36, 51, 65
reparameterisation trick, 24
representation collapse, 60
representation learning, 15, 55

Self-Supervised Learning, 55
self-supervised learning, 55
silhouette score, 47
SimCLR, 55, 59
SimSiam, 60
softmax confidence, 46

Uncertainty Estimation, 39
uncertainty estimation, 39

VAE, 15, 22
  anomaly detection, 45
Variational Autoencoder, 15
variational inference, 15
VQ-VAE, 23

Z-score, 44